



USC Viterbi
School of Engineering

Memory MGMT: LRU, CAMP

Shahram Ghandeharizadeh
CSCI 585



Divide-And-Conquer

When designing an algorithm, you are a 5-star General going to war.

A general has a choice:

- **Win** by dividing and conquering their opponent, OR
- **Lose** by dividing his army.



Divide-And-Conquer

When designing an algorithm,
you are a 5-star General going
to war.

You have a choice:

- **Win** by dividing and conquering *the problem*,
OR
- **Lose** by dividing your
resources.



Node Hardware & Disk I/O

Hardware Organization

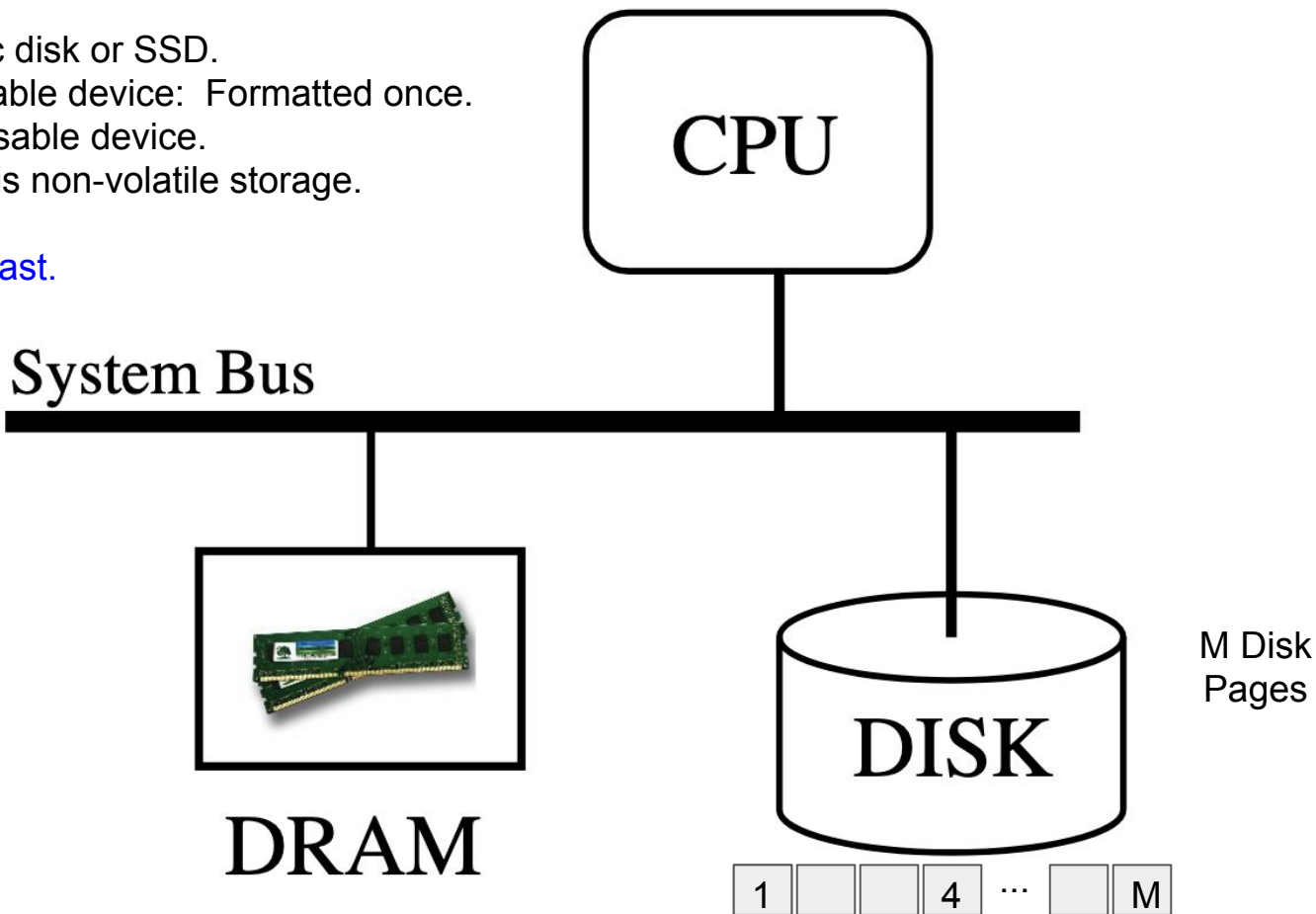
Disk may be a magnetic disk or SSD.

Disk is a block addressable device: Formatted once.

DRAM is a byte-addressable device.

DRAM is volatile. Disk is non-volatile storage.

Disk is slow. DRAM is fast.

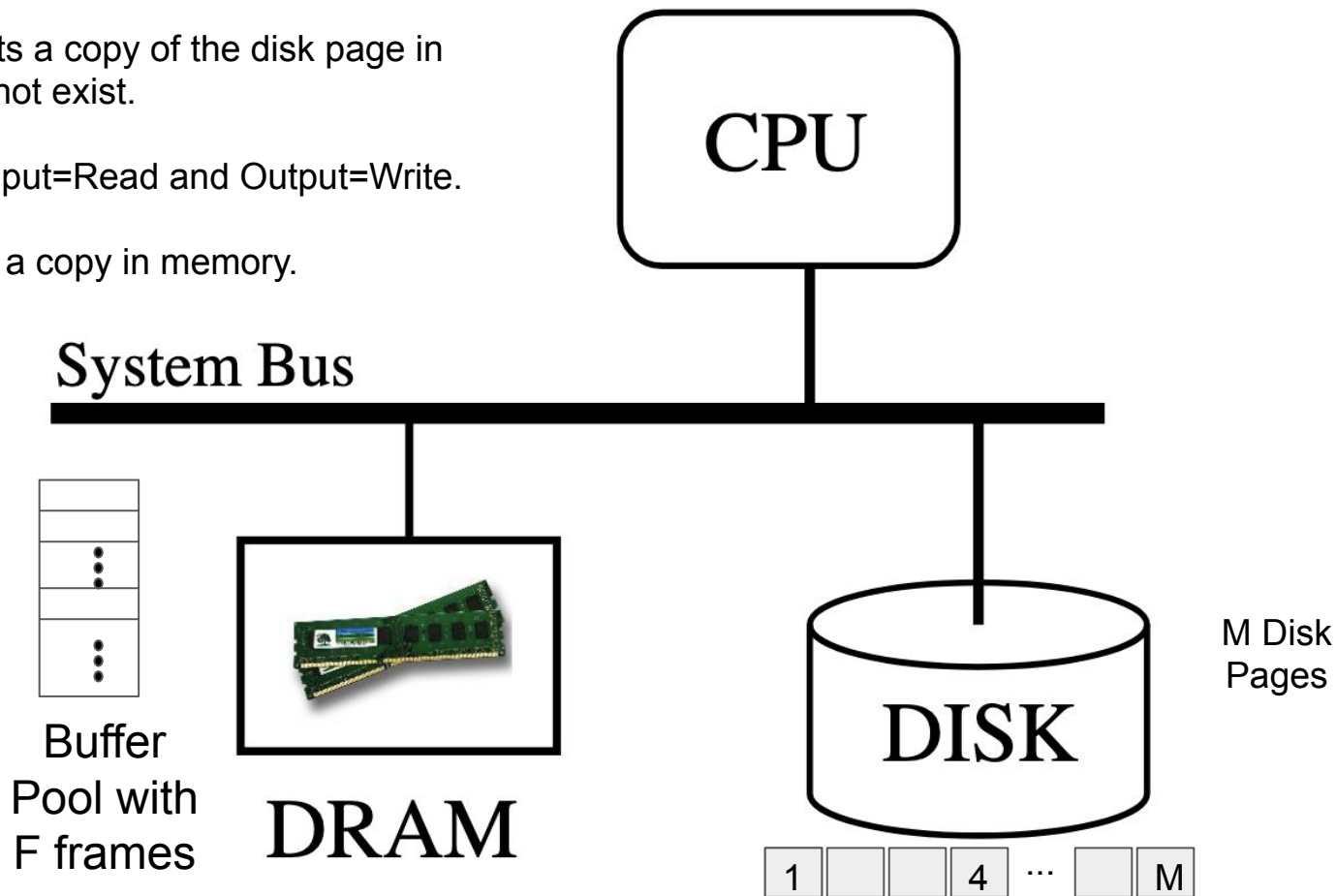


Disk Input/Output (I/O)

A Disk read constructs a copy of the disk page in memory if one does not exist.

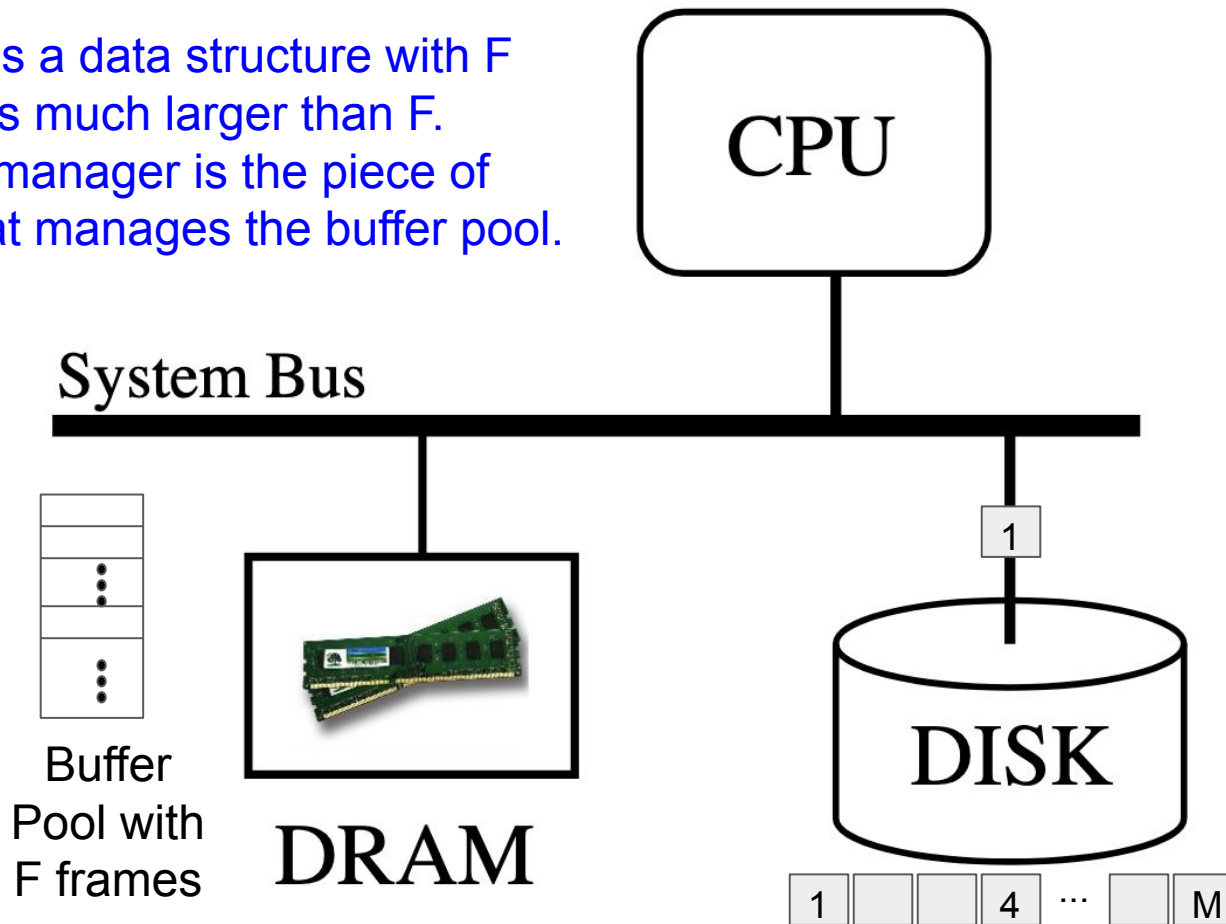
Disk I/O stands for Input=Read and Output=Write.

Disk write may leave a copy in memory.



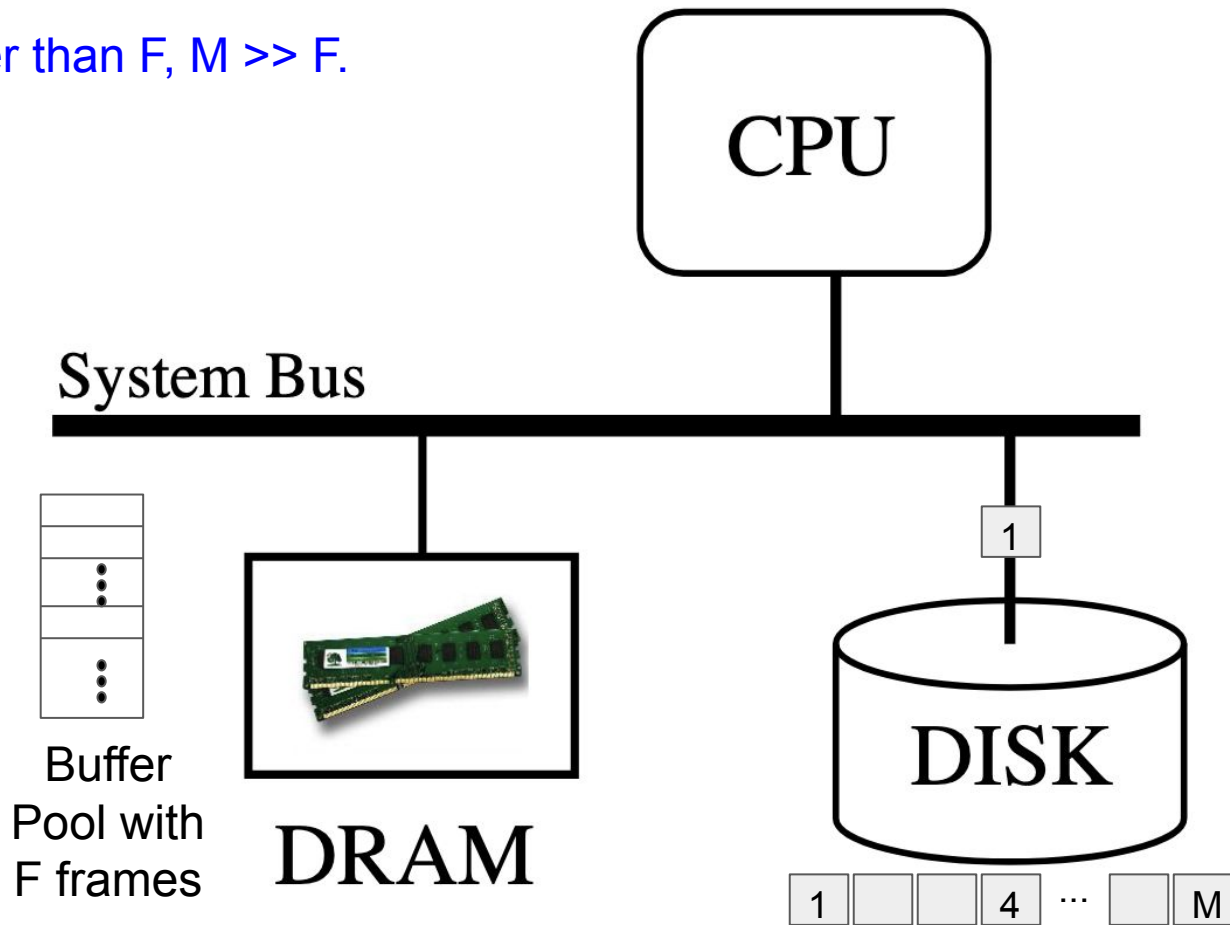
Disk Input/Output (I/O): Terminology

- Buffer pool is a data structure with F frames. M is much larger than F .
- Buffer pool manager is the piece of software that manages the buffer pool.

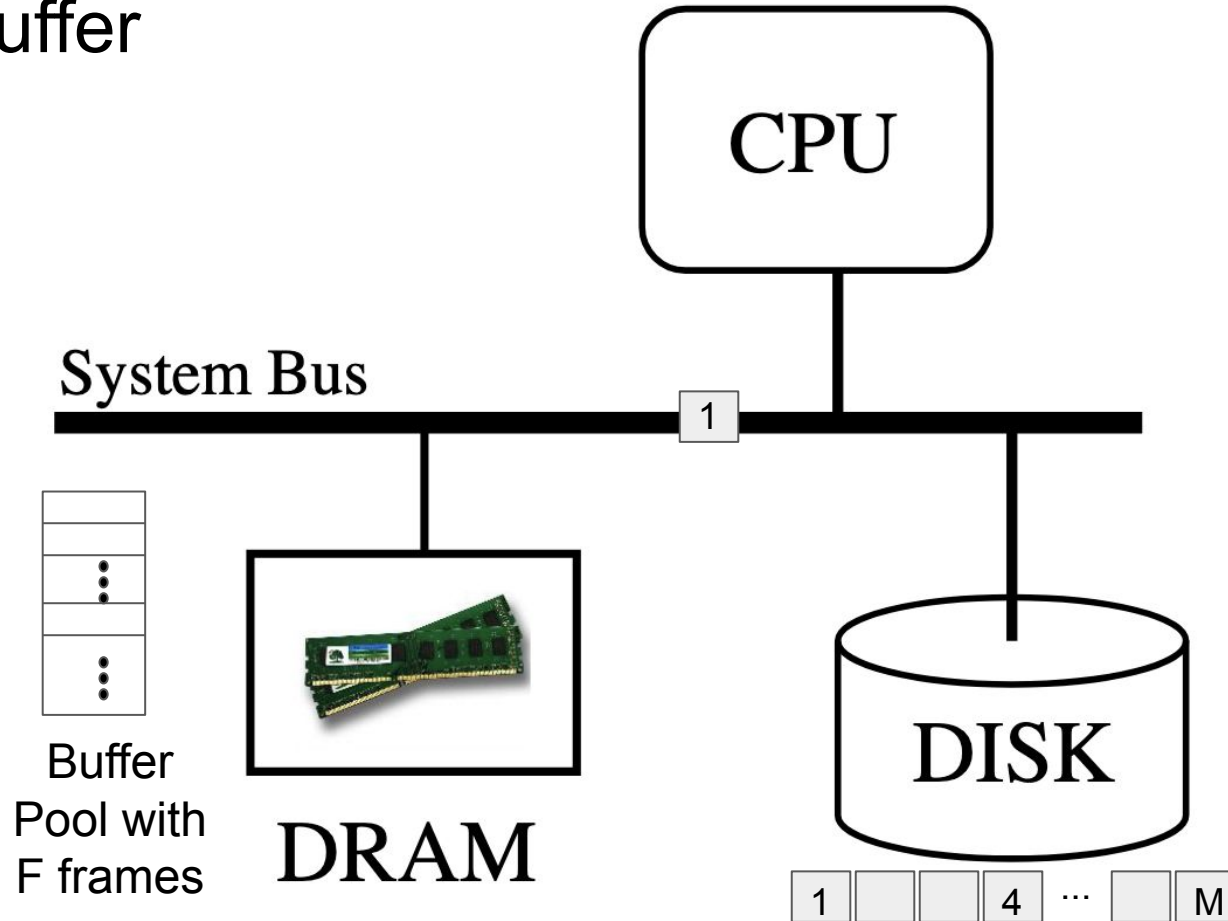


Disk Input/Output (I/O): Read Disk Page 1

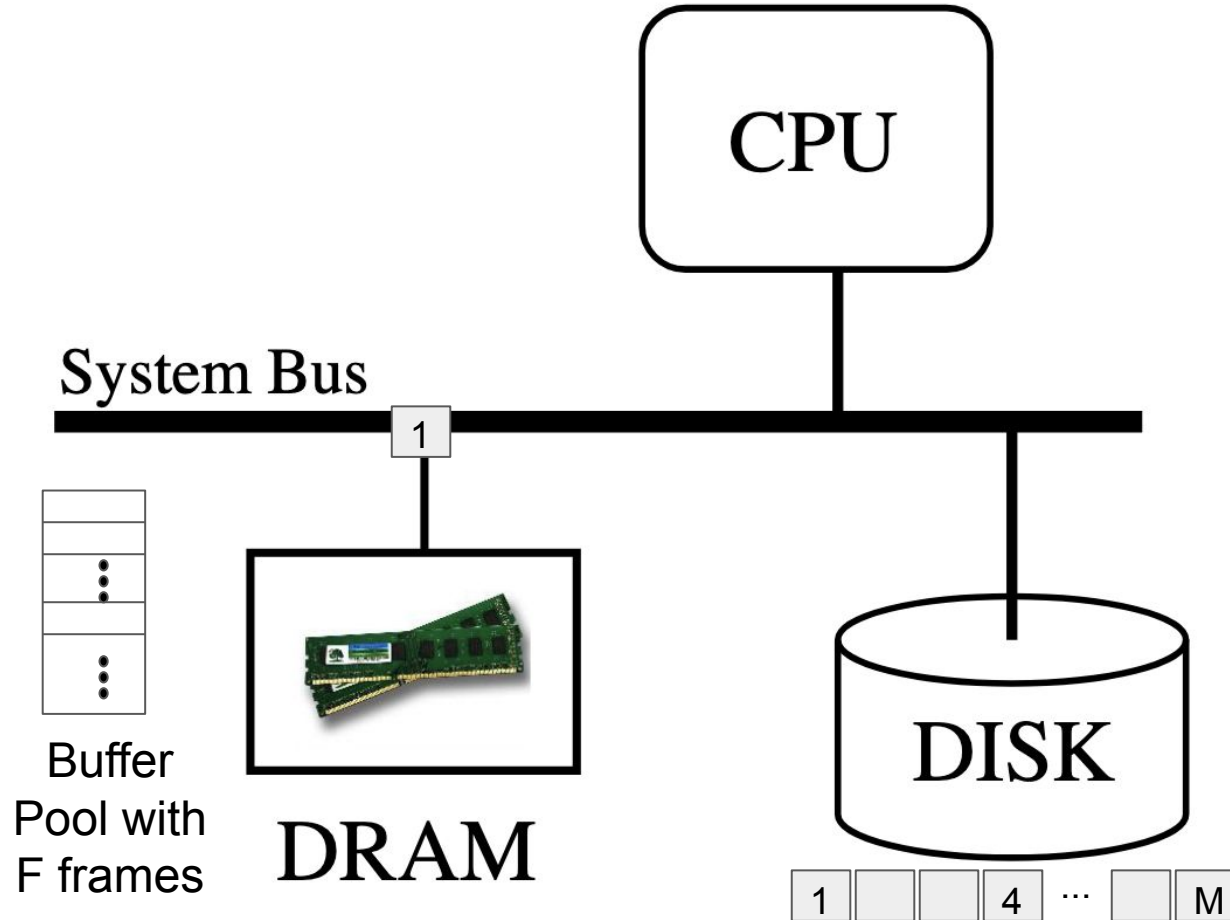
M is much larger than F , $M \gg F$.



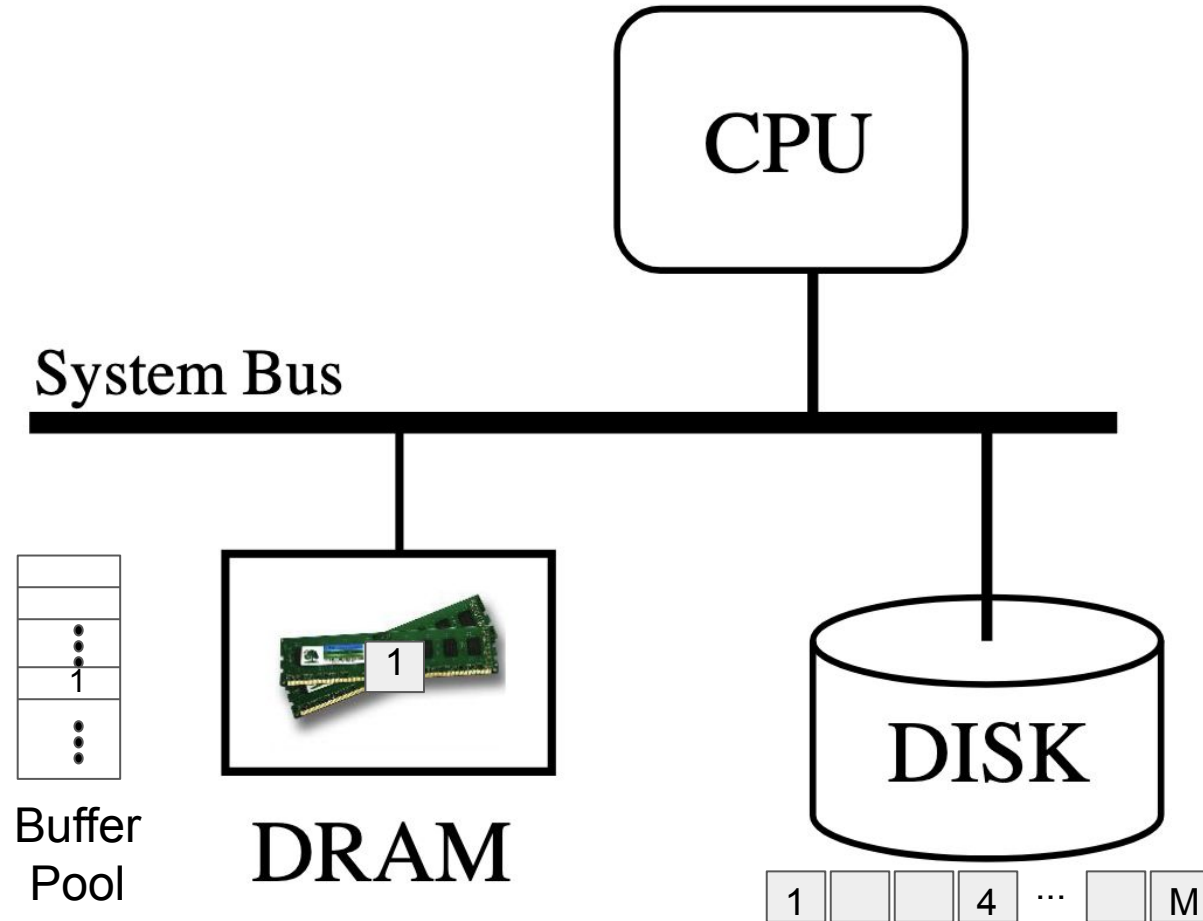
Disk Input/Output (I/O): Page 1 is Written into a Memory Buffer



Disk Input/Output (I/O)

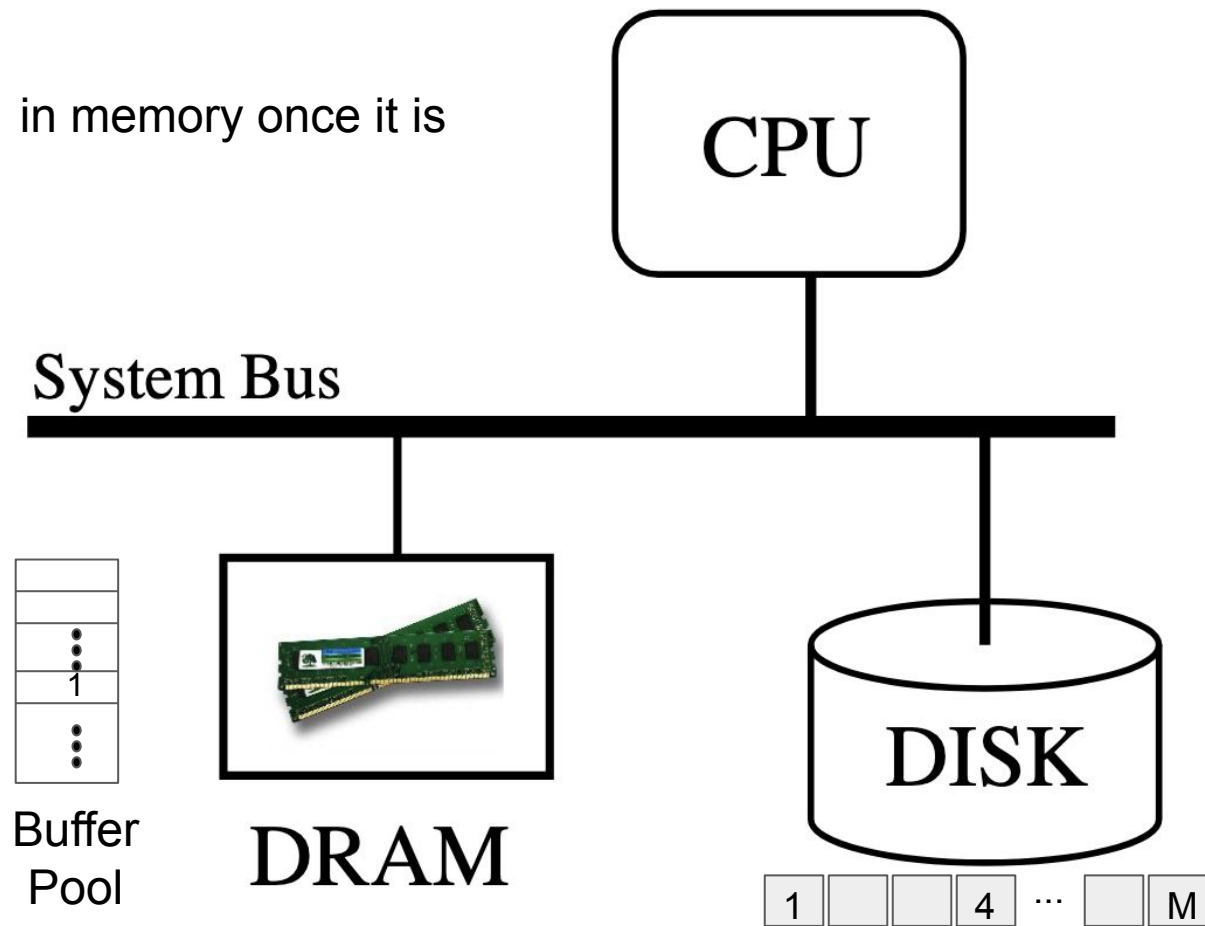


Buffer Pool Consists of Disk Page Frames



A Disk Page Occupies a Frame

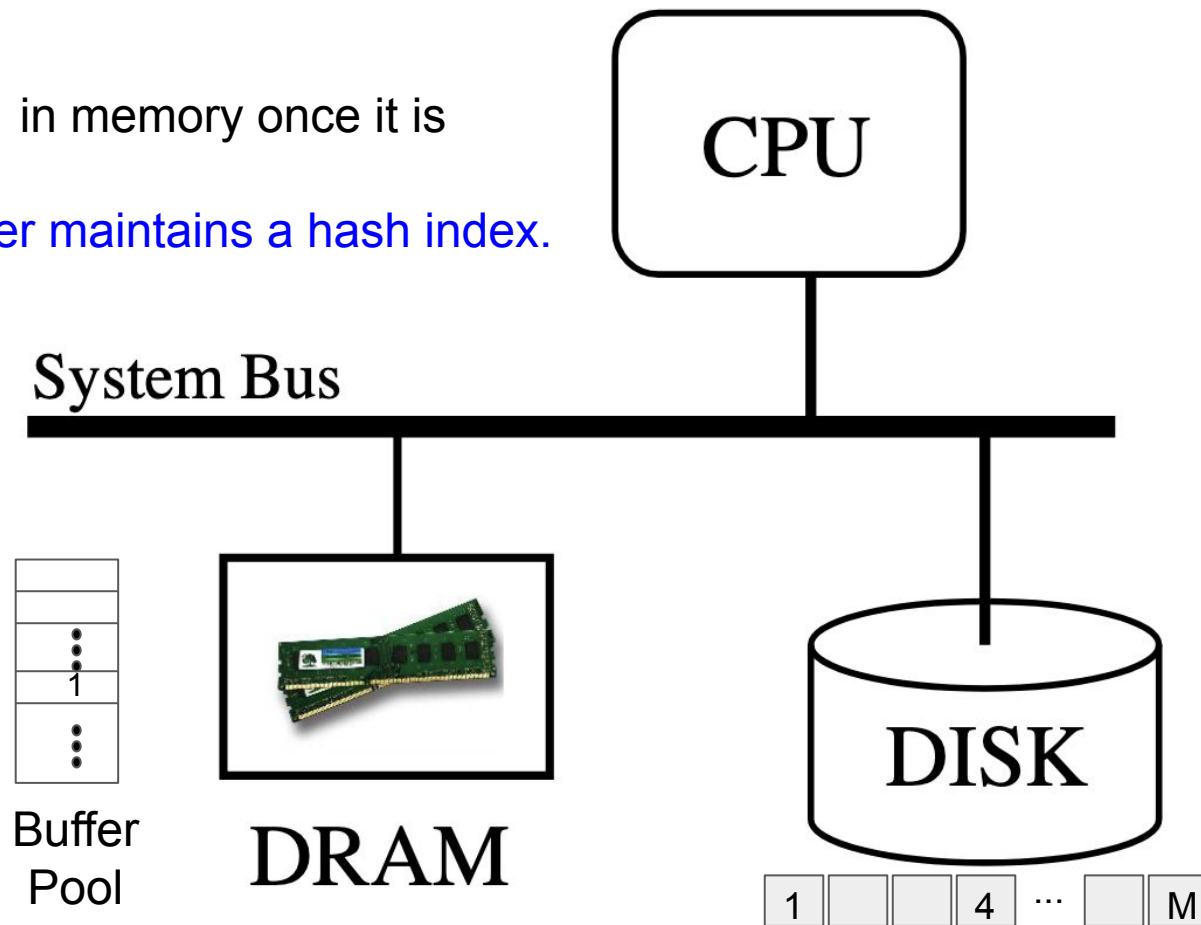
How to find Page 1 in memory once it is referenced?



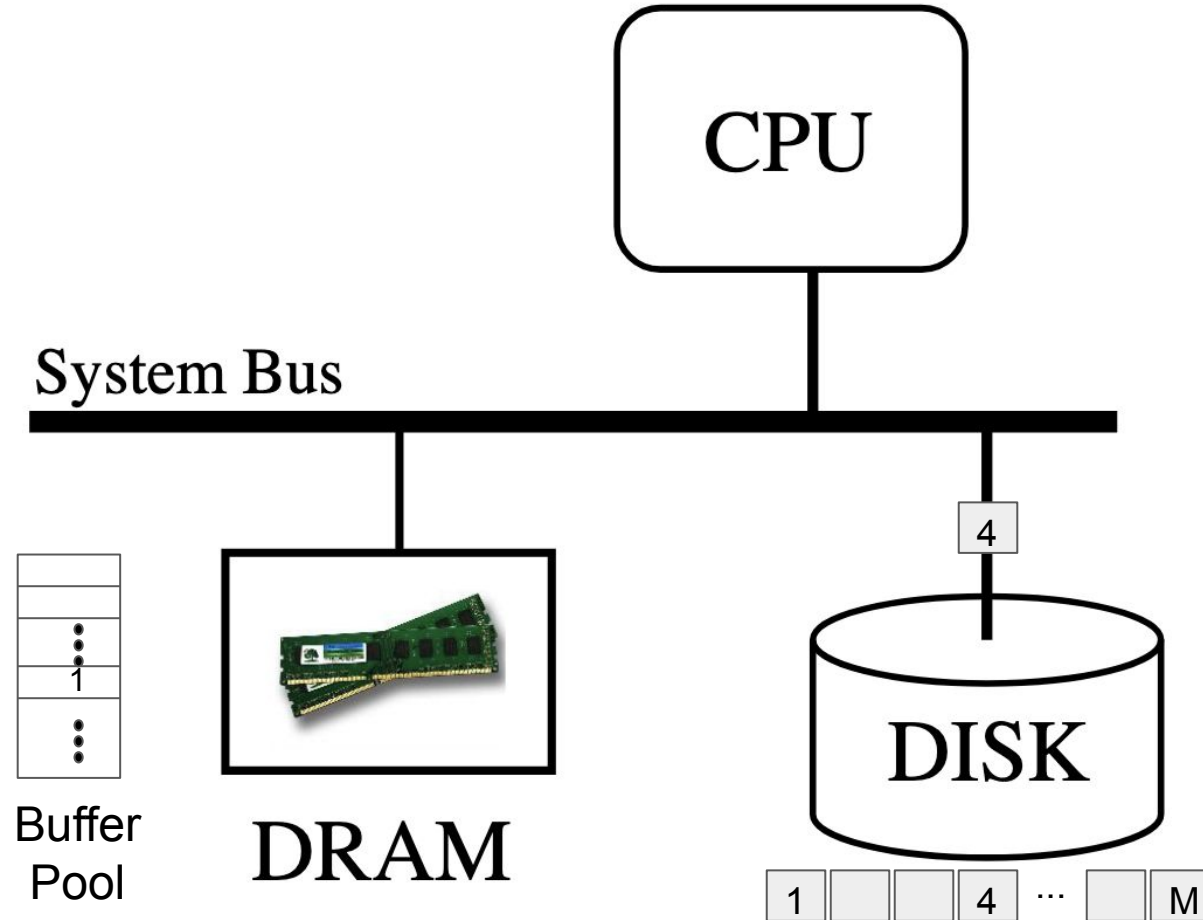
A Disk Page Occupies a Frame

How to find Page 1 in memory once it is referenced?

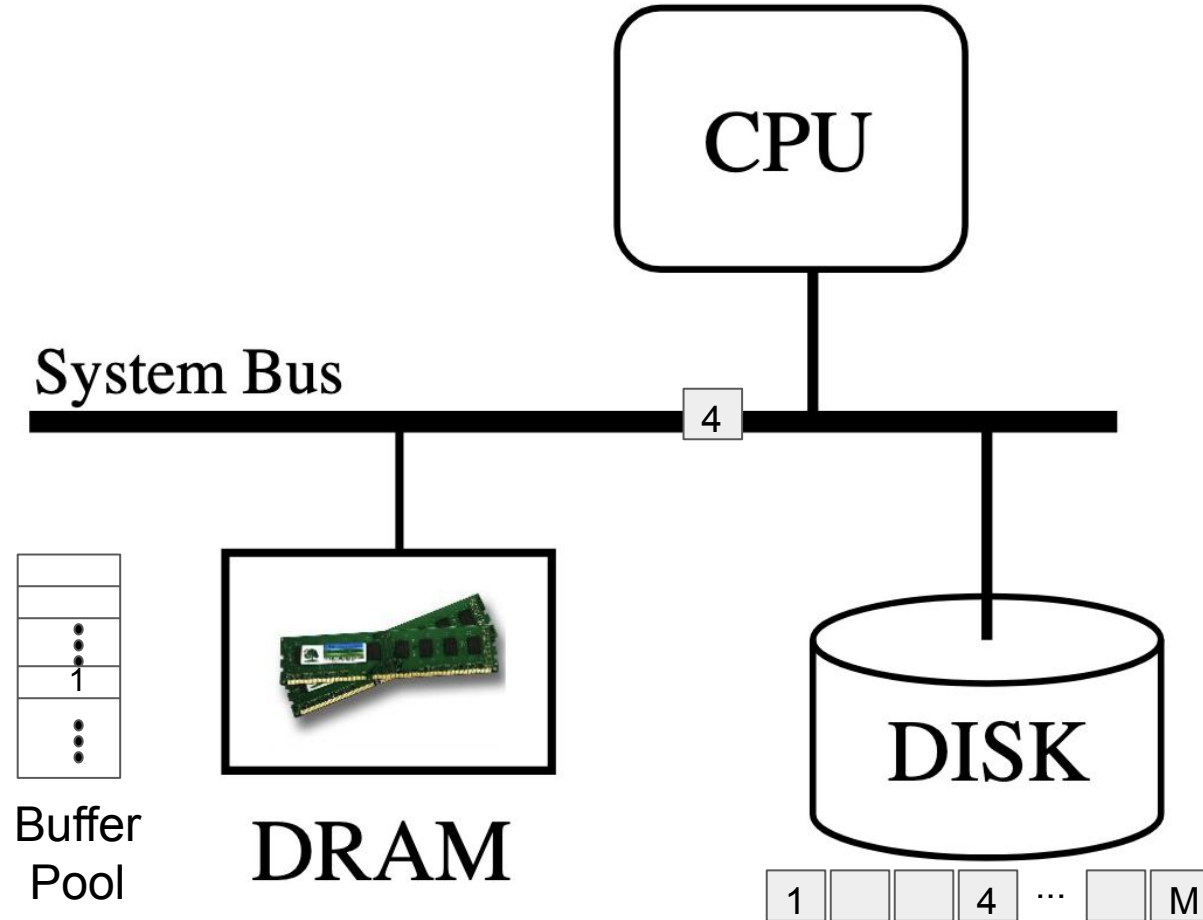
Buffer pool manager maintains a hash index.



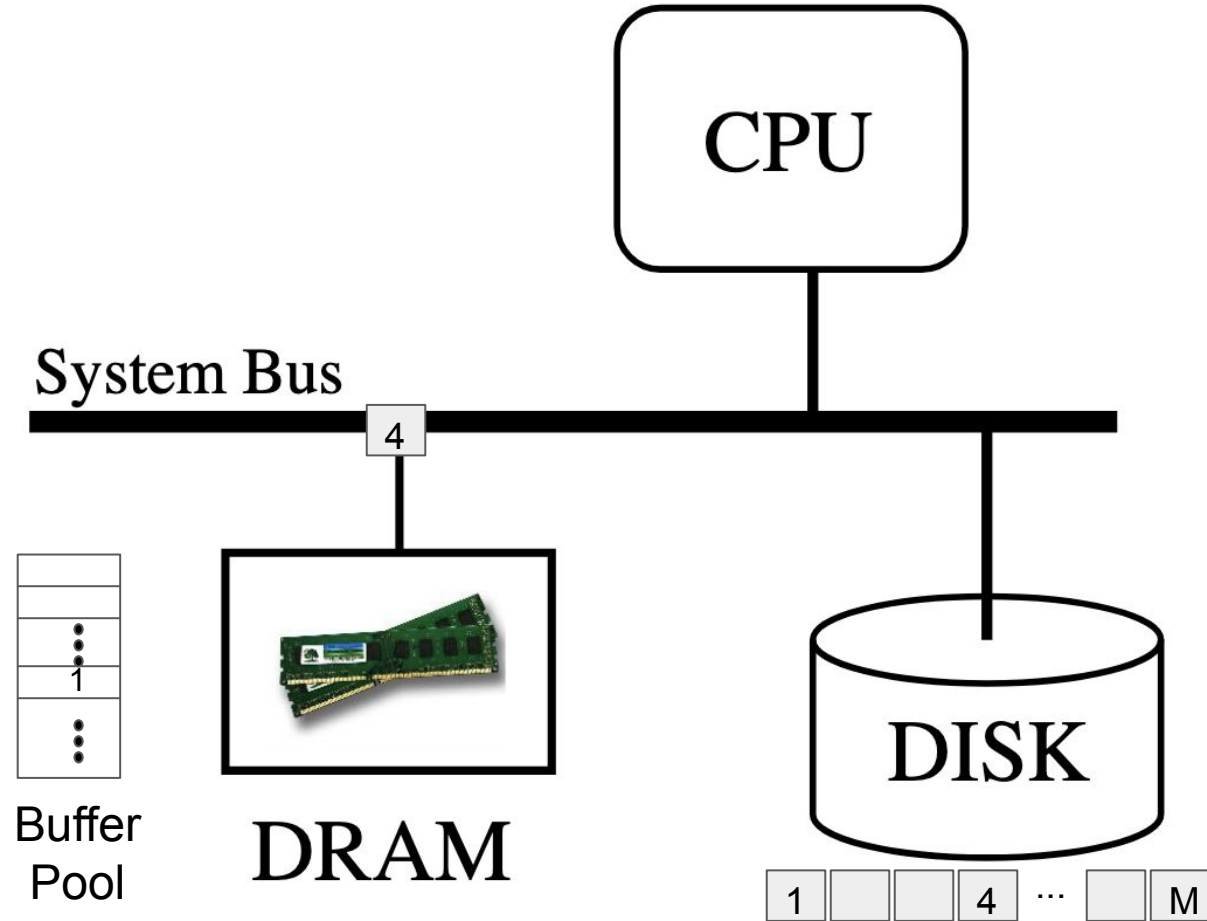
Disk Input/Output (I/O): Read Page 4



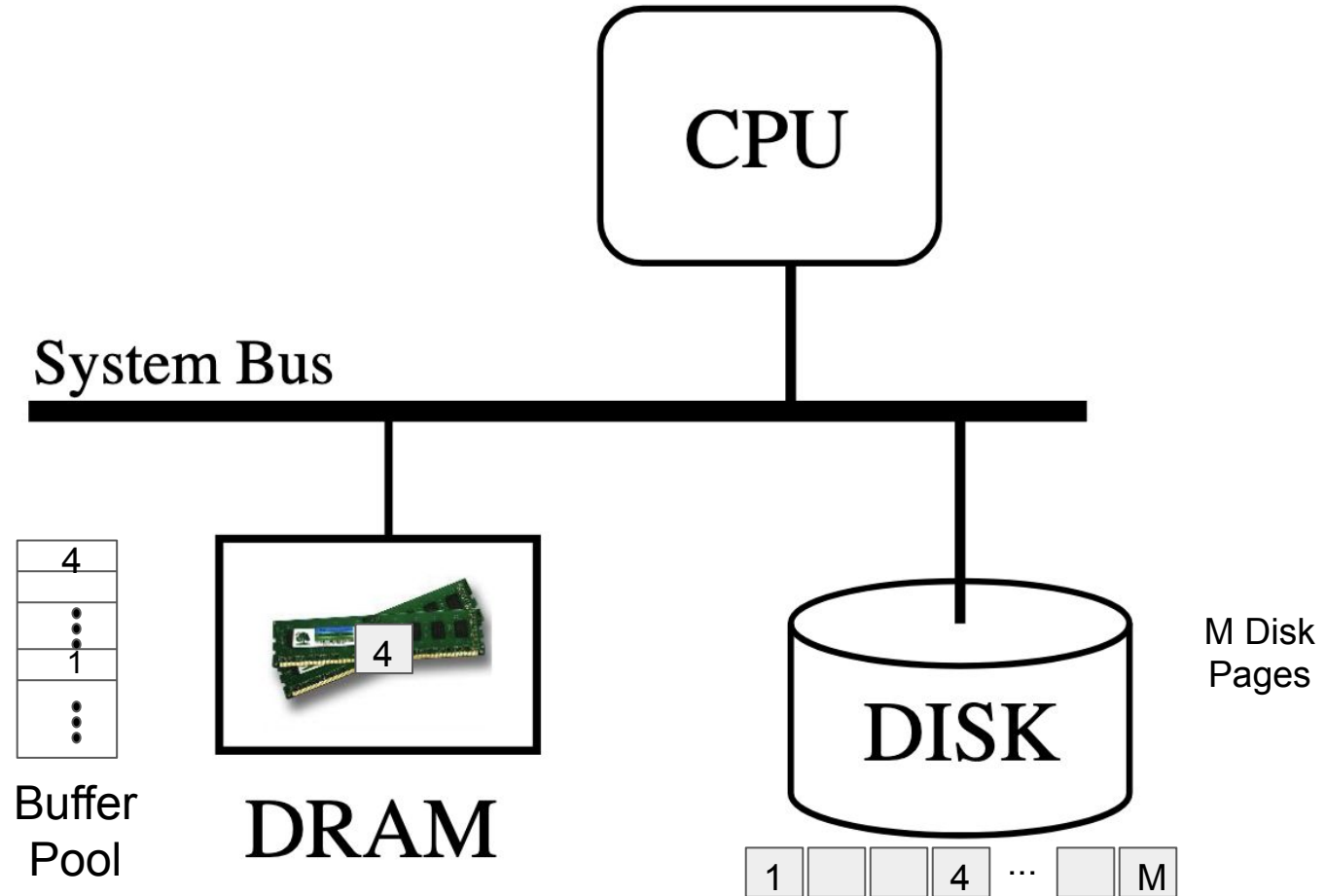
Disk Input/Output (I/O)



Disk Input/Output (I/O)

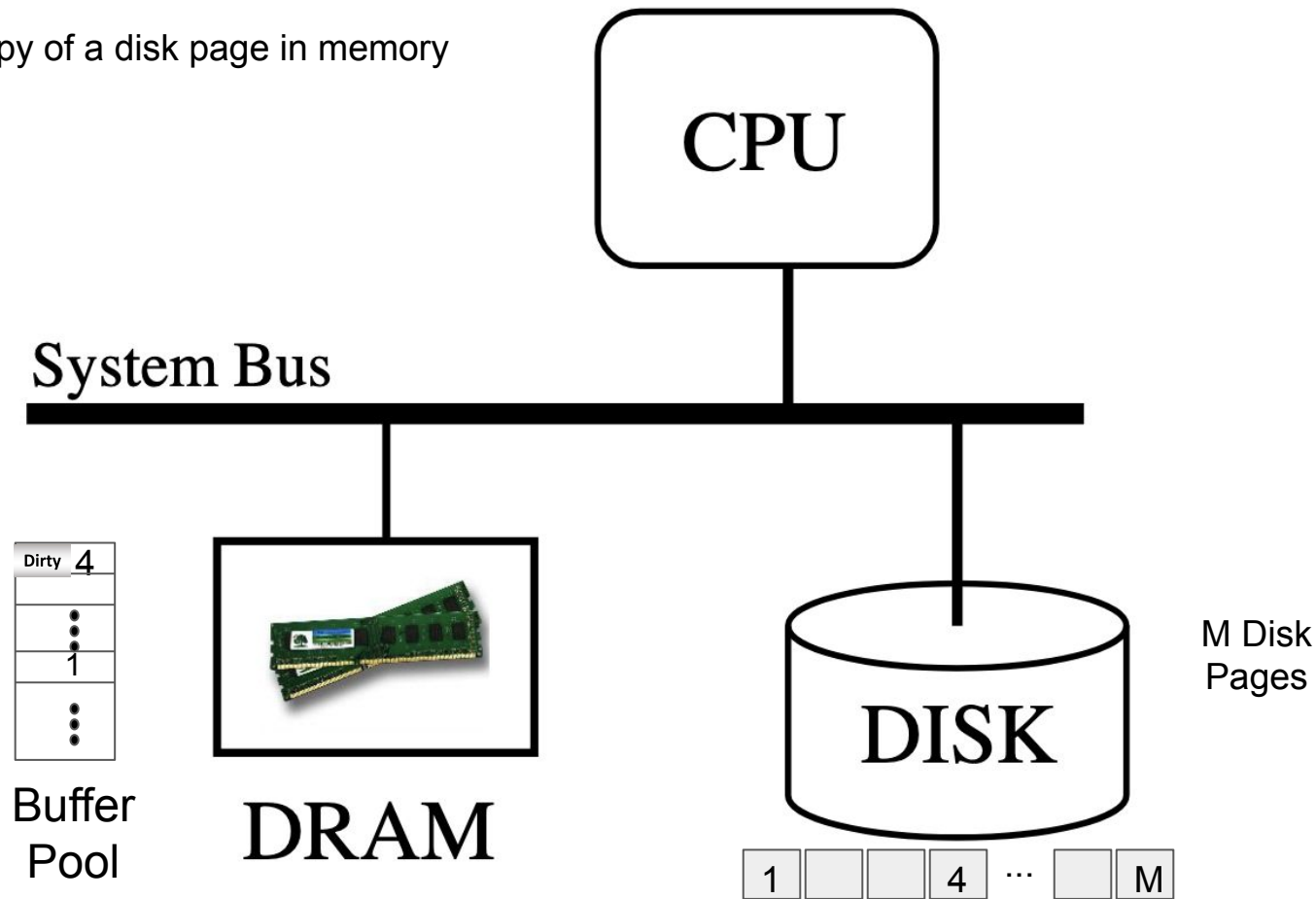


Each Disk Page Occupies a Buffer Pool Frame



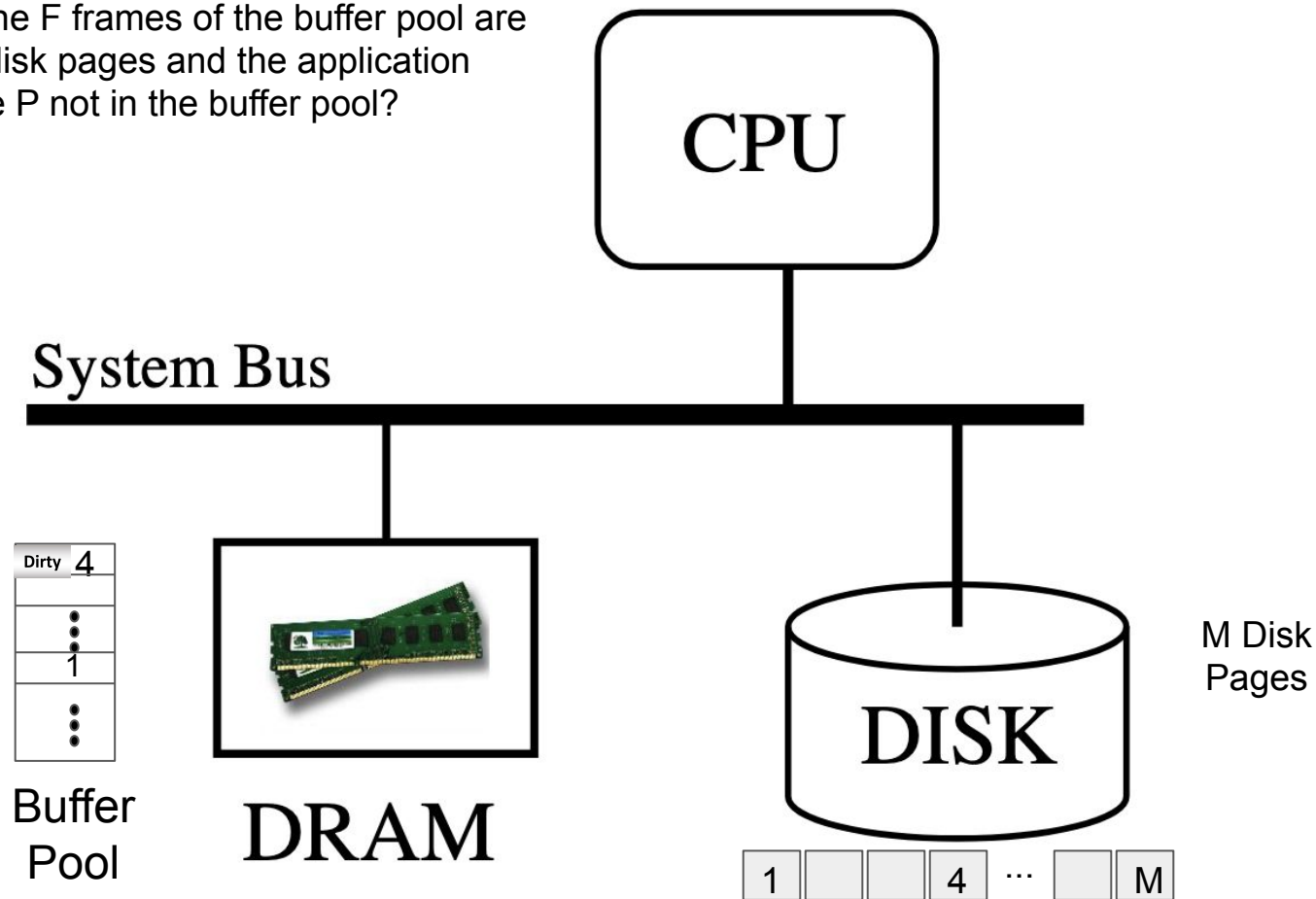
Disk Input/Output (I/O)

A write updates the copy of a disk page in memory and marks it dirty.



Disk Input/Output (I/O)

What happens when the F frames of the buffer pool are occupied by different disk pages and the application references a disk page P not in the buffer pool?

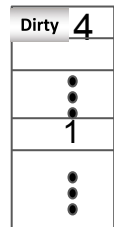


Disk Input/Output (I/O)

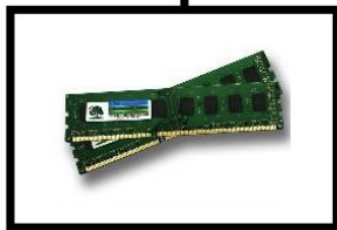
What happens when the F frames of the buffer pool are occupied by different disk pages and the application references a disk page P not in the buffer pool?

- Select a page in the buffer pool to evict. This is the victim page.
- If the victim page is dirty then write it to disk.
- Free the frame.
- Read page P in the freed frame.

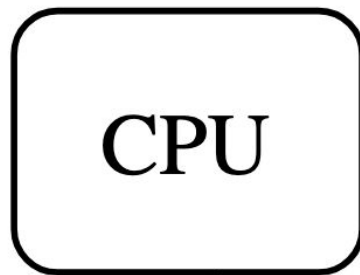
System Bus



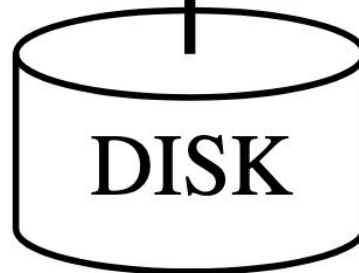
Buffer
Pool



DRAM



CPU



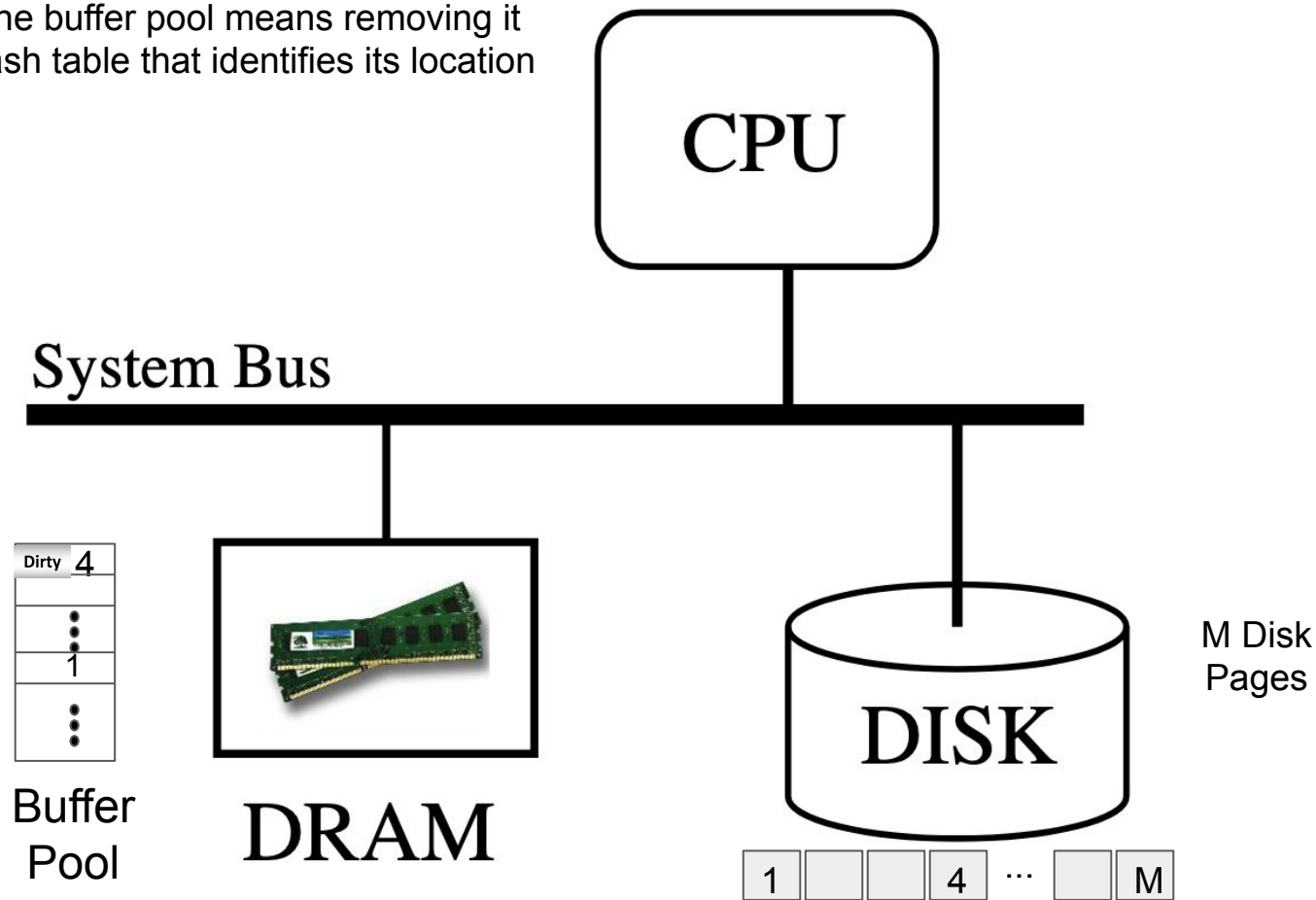
DISK

M Disk
Pages



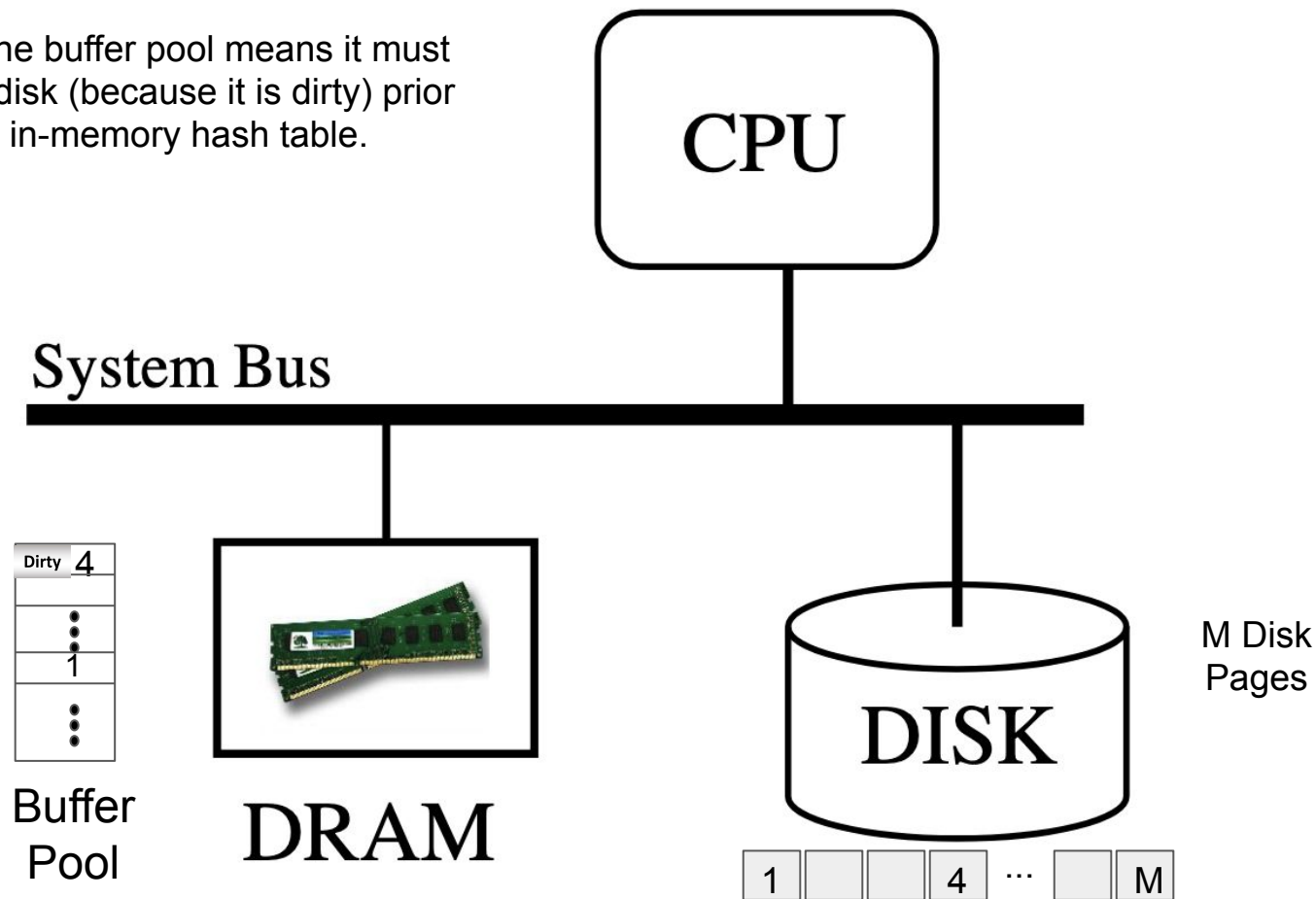
Disk Input/Output (I/O)

Evicting Page 1 from the buffer pool means removing it from the in-memory hash table that identifies its location in memory.



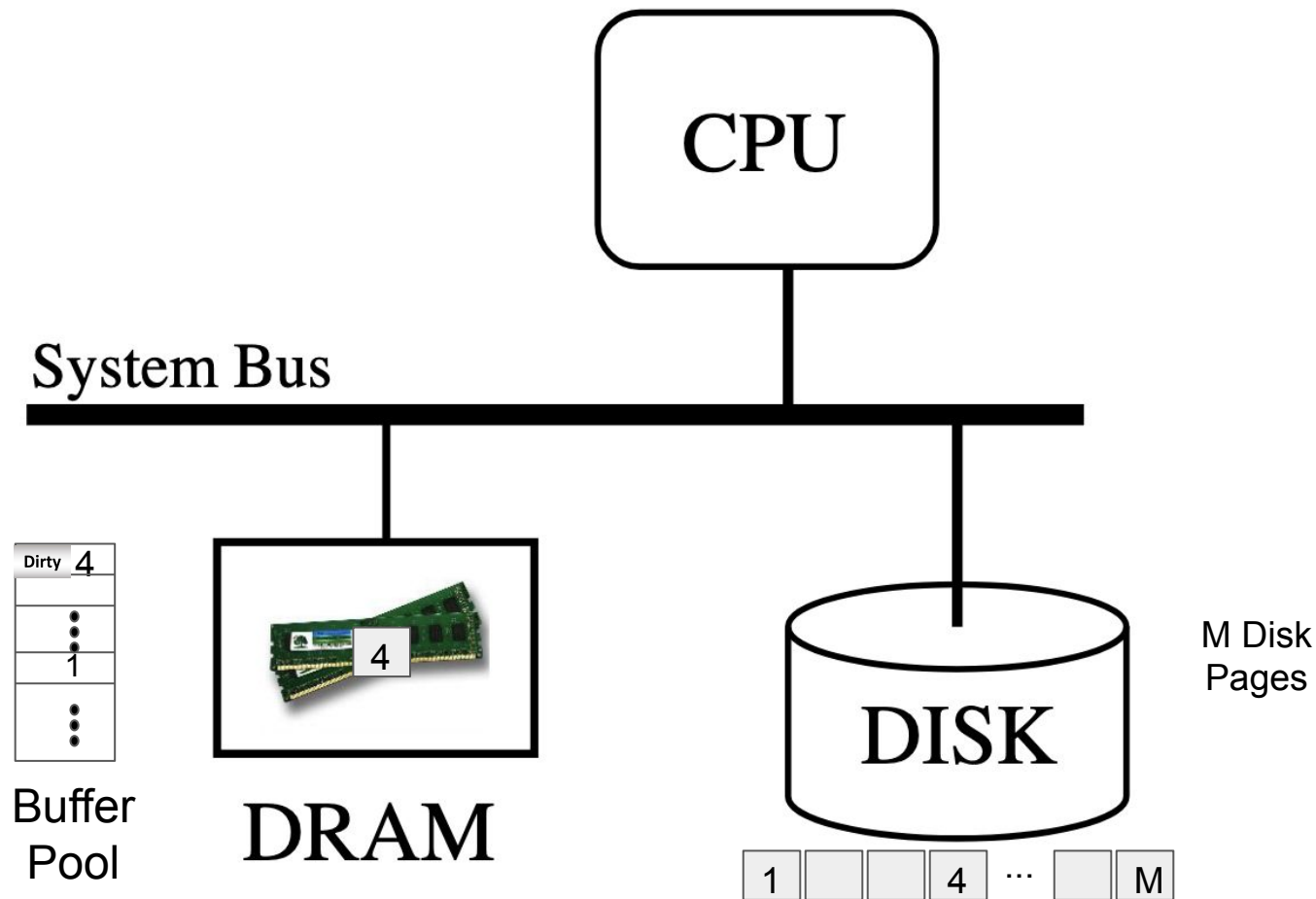
Disk Input/Output (I/O)

Evicting Page 4 from the buffer pool means it must be written back to the disk (because it is dirty) prior to removing it from the in-memory hash table.

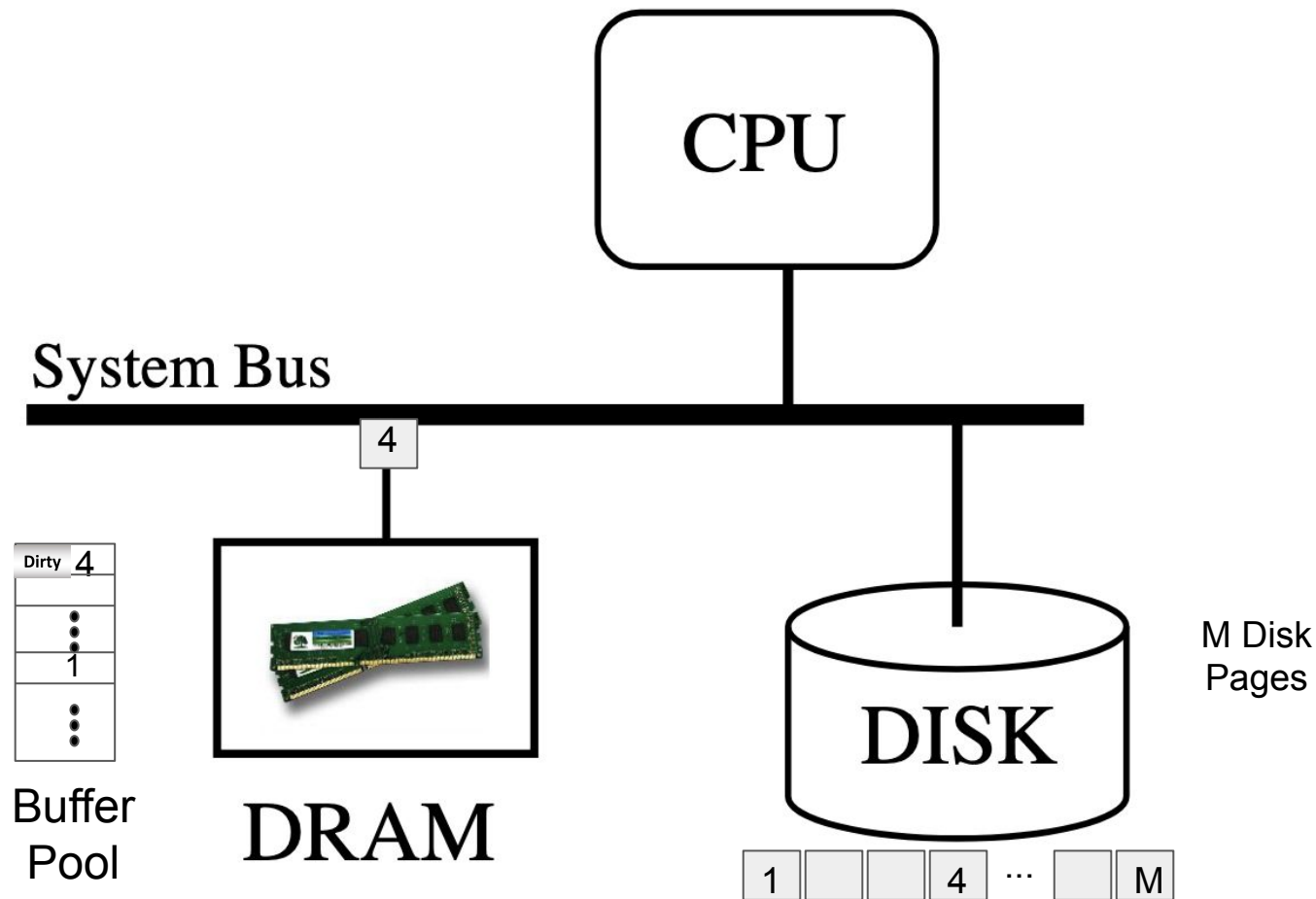


Disk Input/Output (I/O)

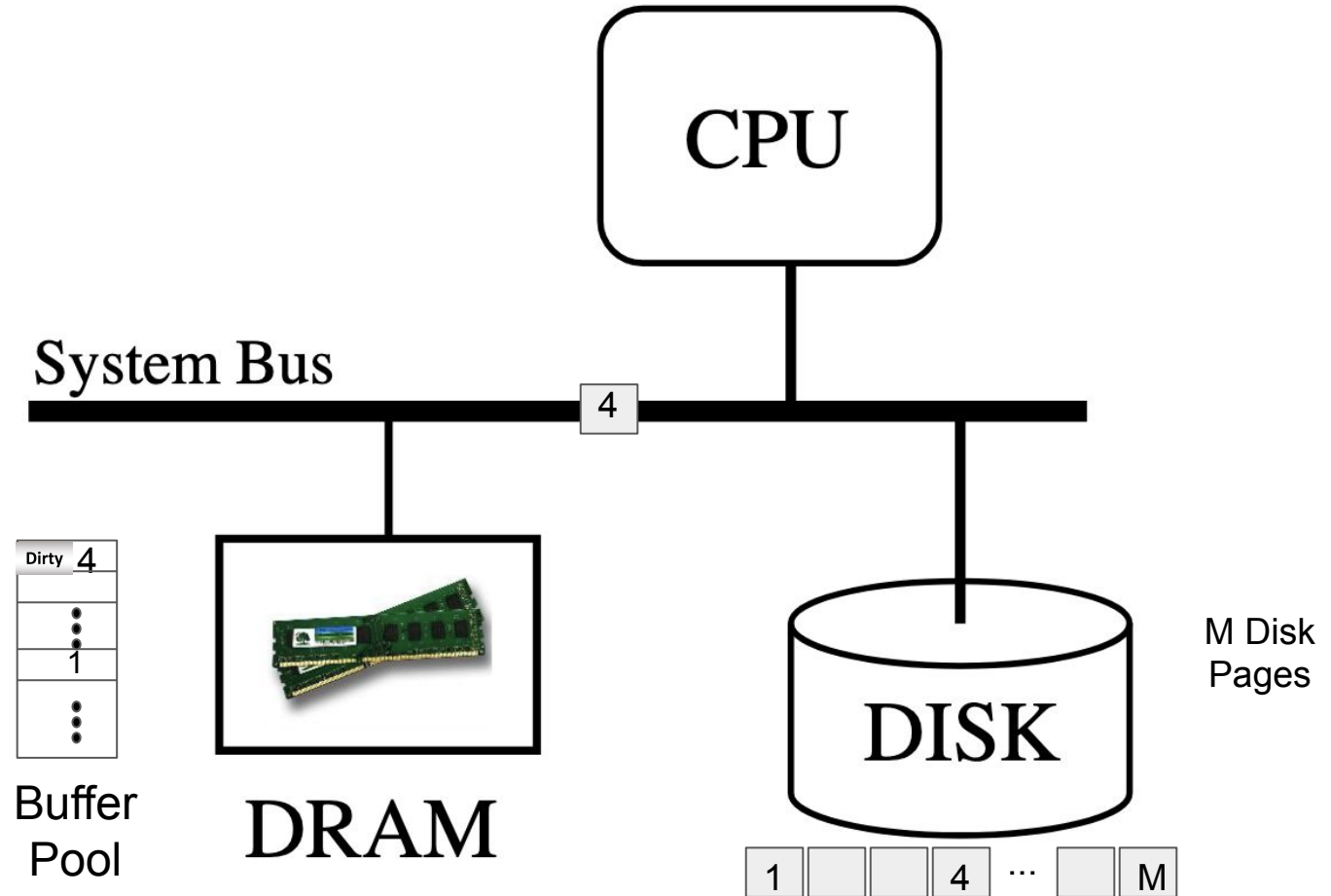
Write Page 4 to Disk.



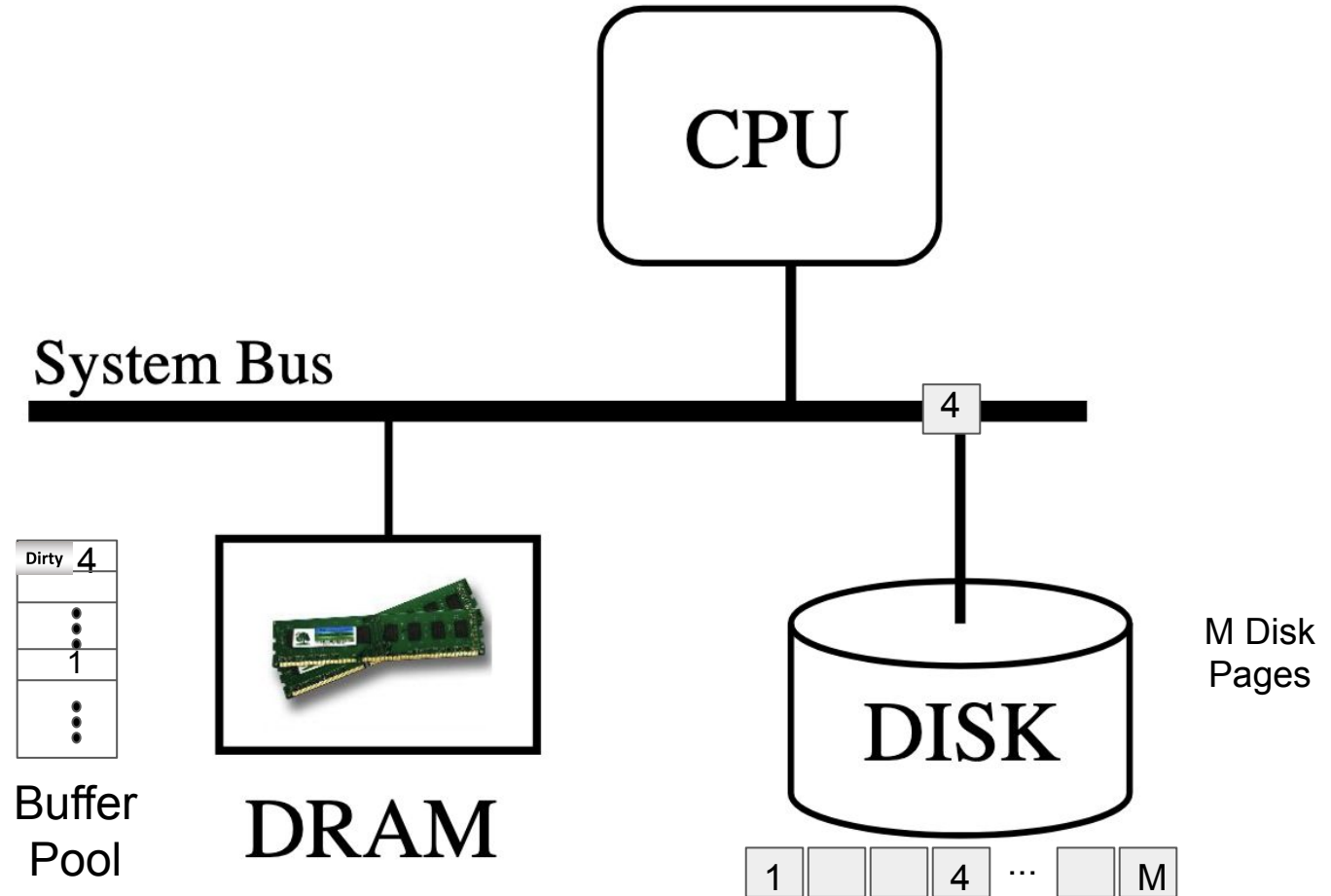
Disk Input/Output (I/O): Flush Page 4 to Disk



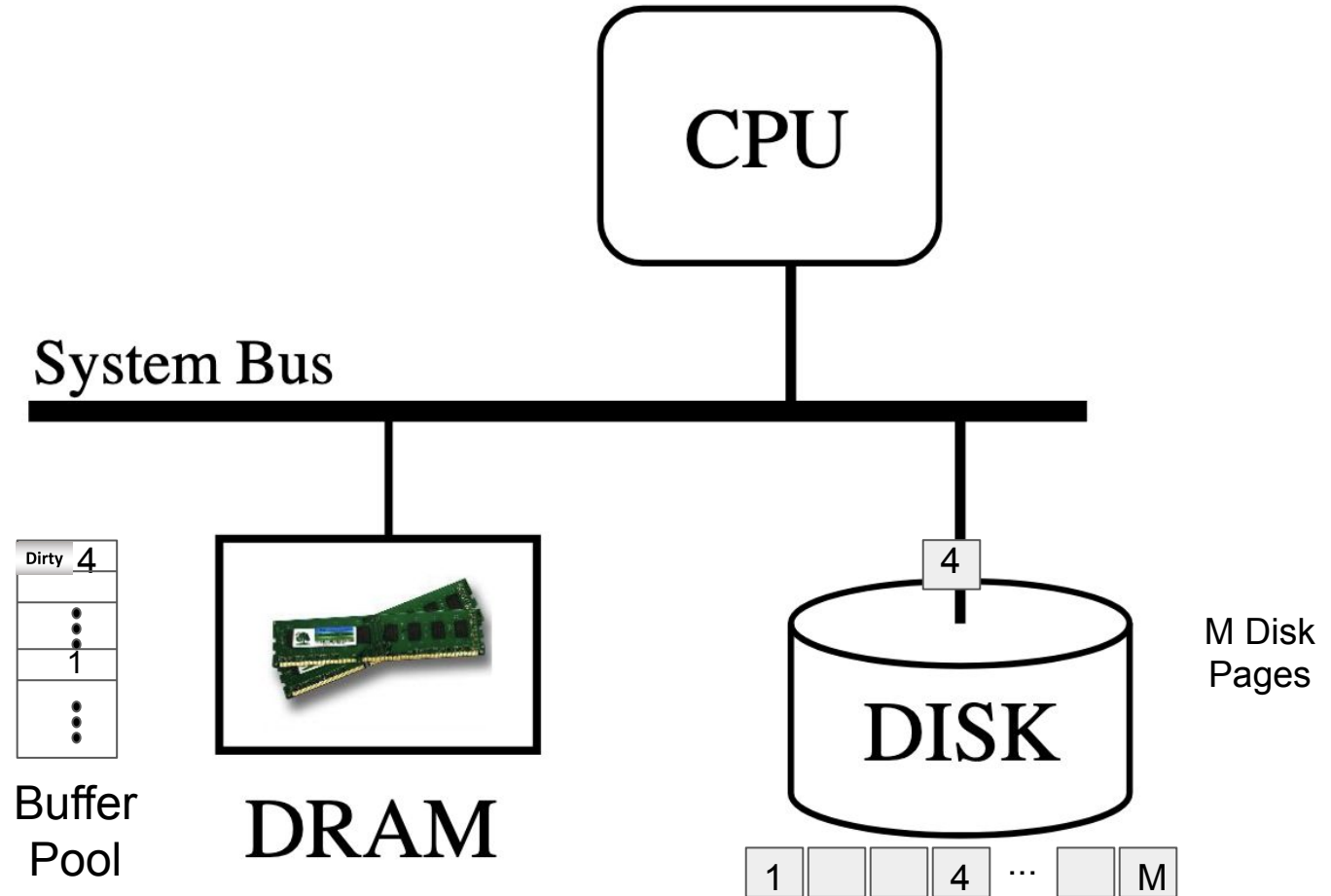
Disk Input/Output (I/O): Flush Page 4 to Disk



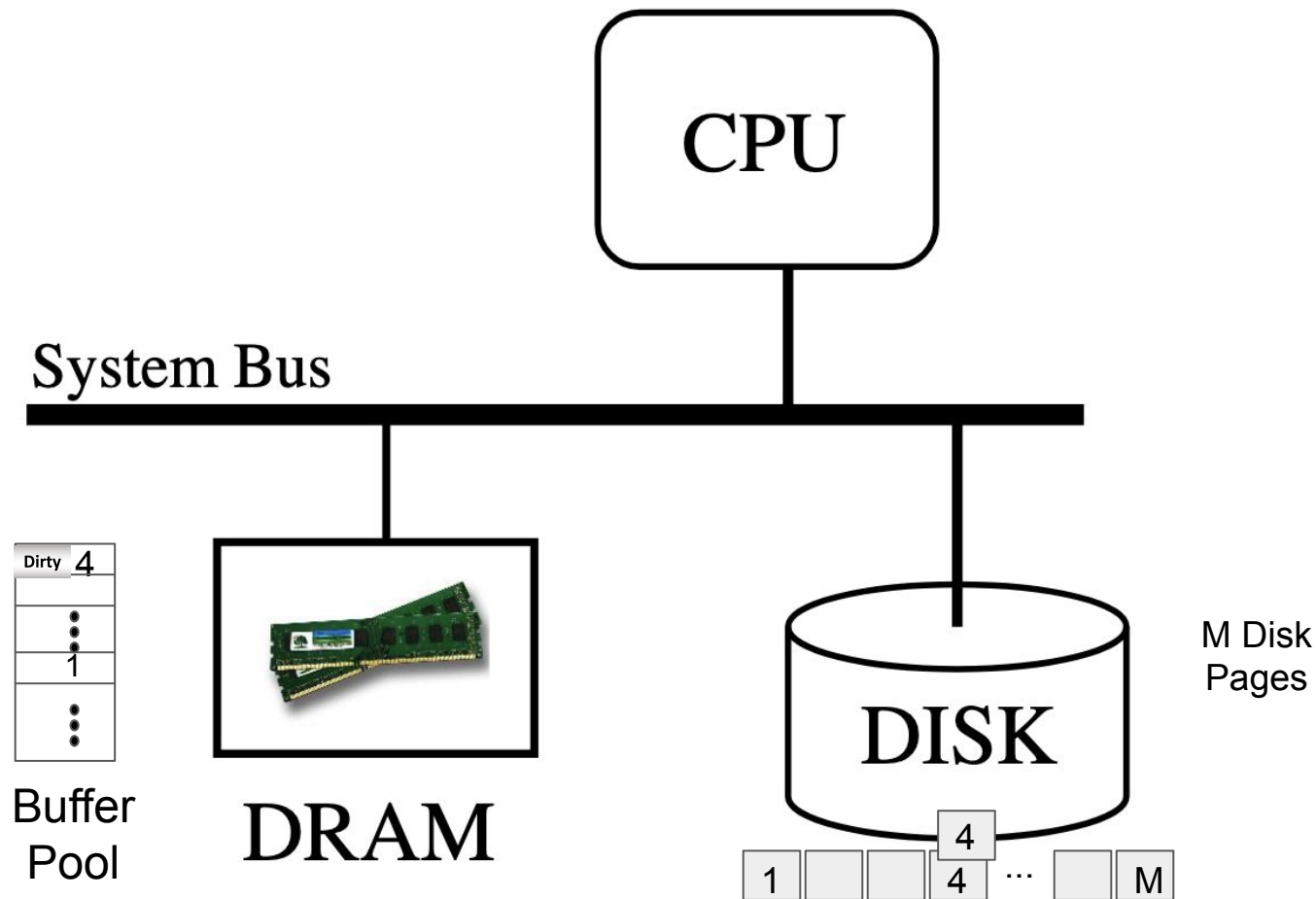
Disk Input/Output (I/O): Flush Page 4 to Disk



Disk Input/Output (I/O): Flush Page 4 to Disk



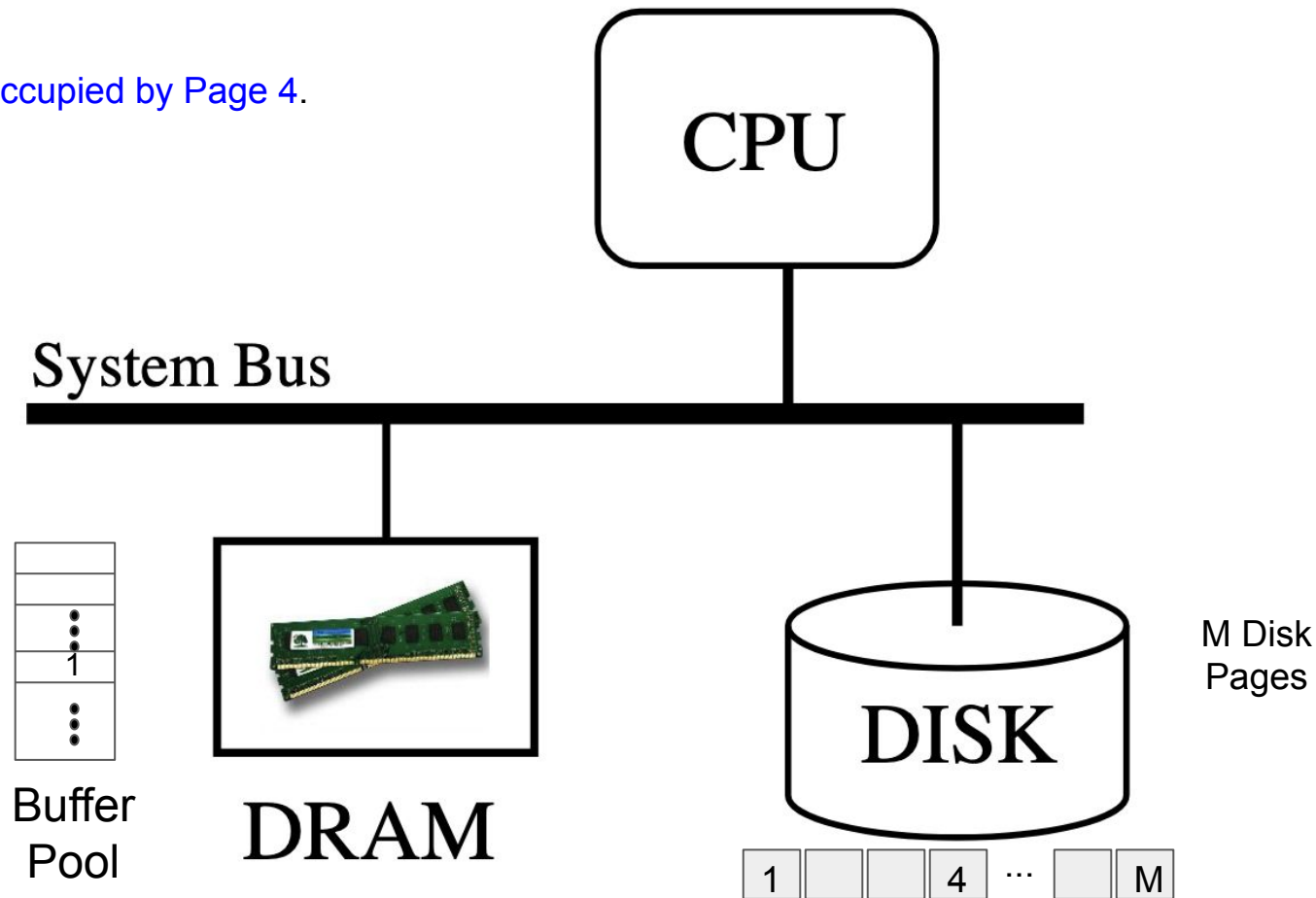
Disk Input/Output (I/O): Flush Page 4 to Disk



Disk Input/Output (I/O): Free the Frame

Flush complete:

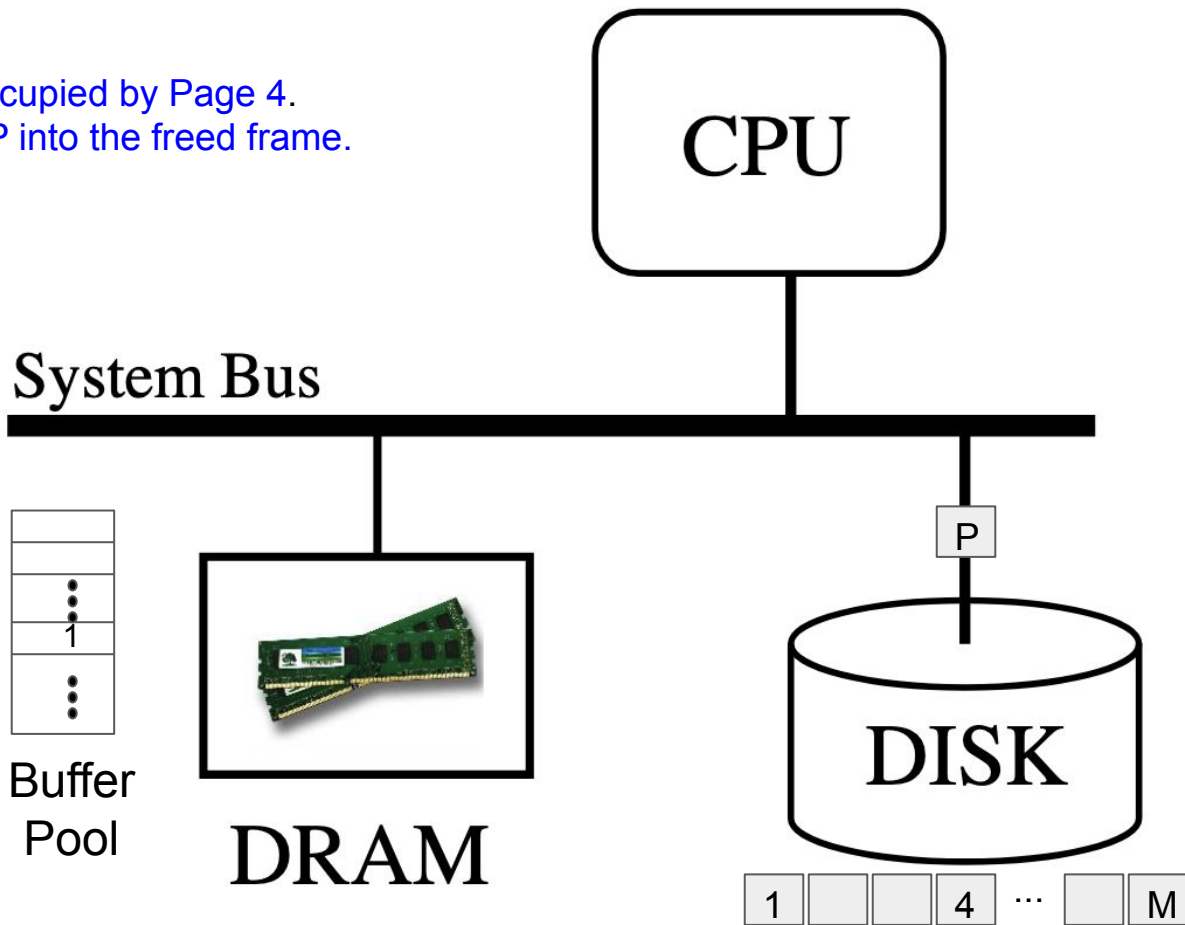
- Free the frame occupied by Page 4.



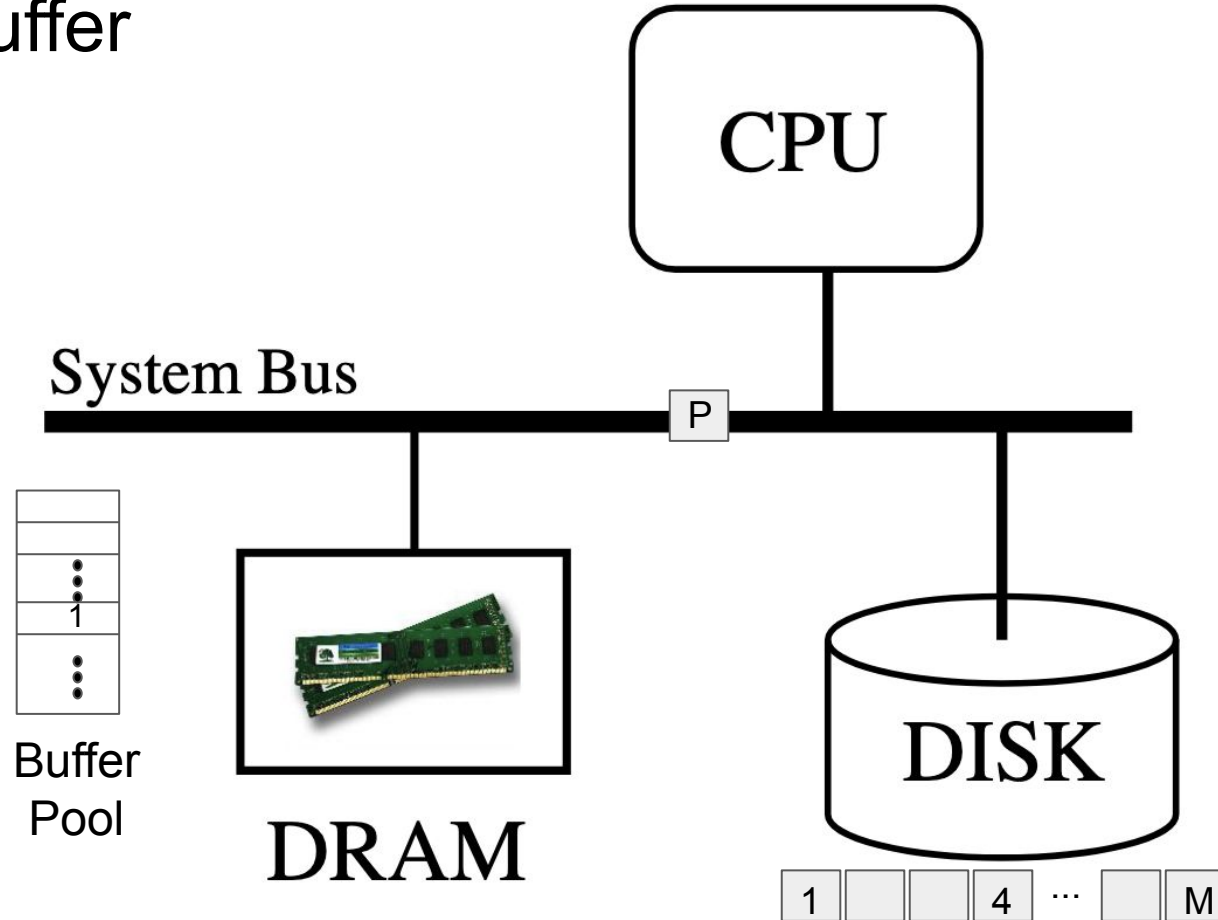
Disk Input/Output (I/O): Read Disk Page P

Flush complete:

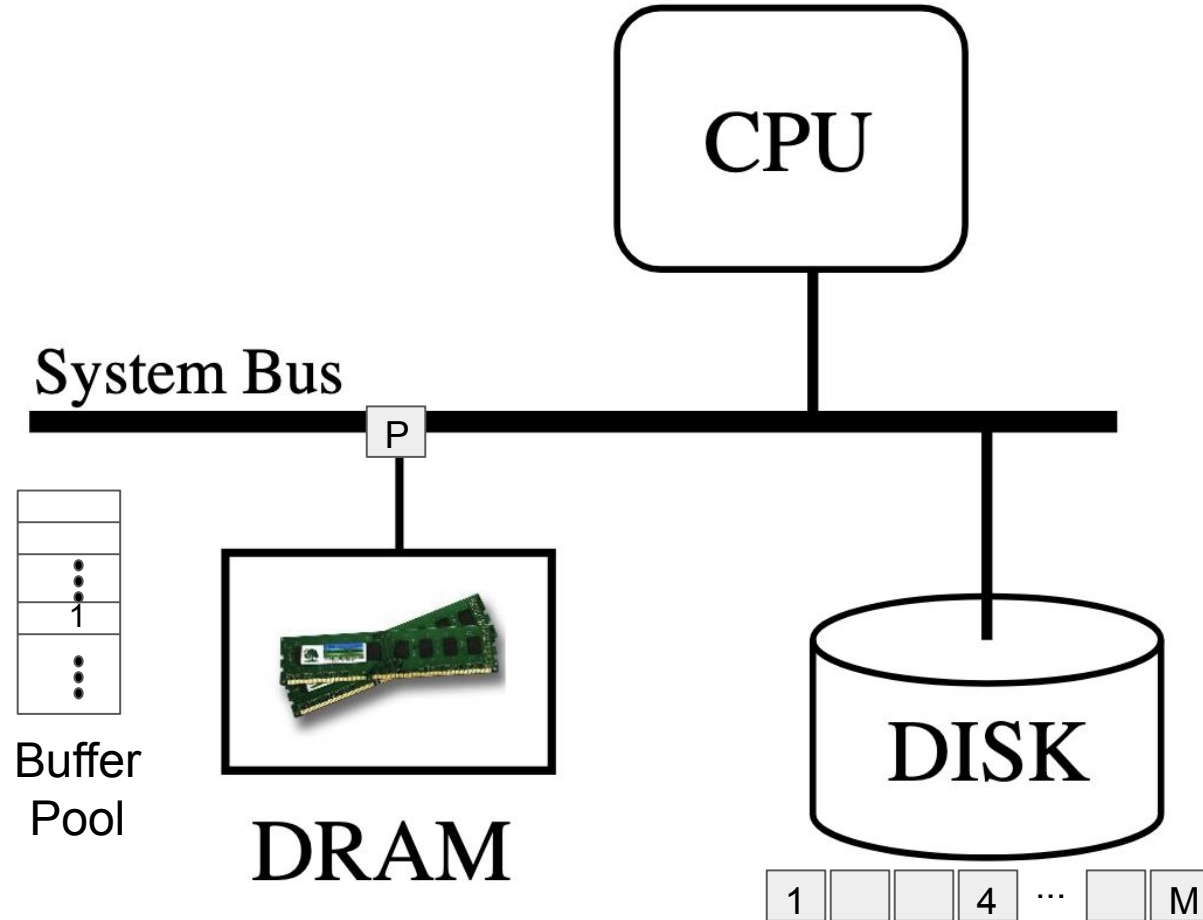
- Free the frame occupied by Page 4.
- Read Disk Page P into the freed frame.



Disk Input/Output (I/O): Page P is Written into a Memory Buffer

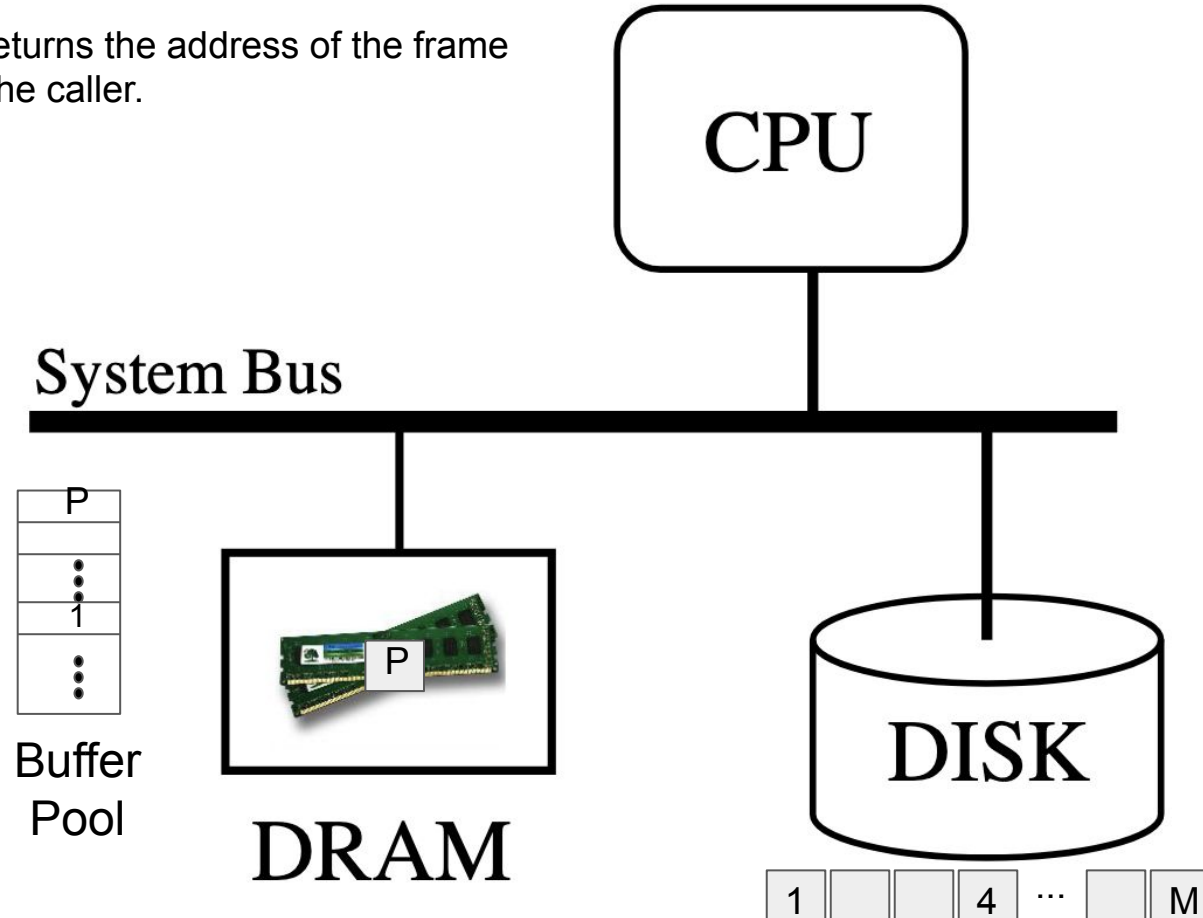


Disk Input/Output (I/O)



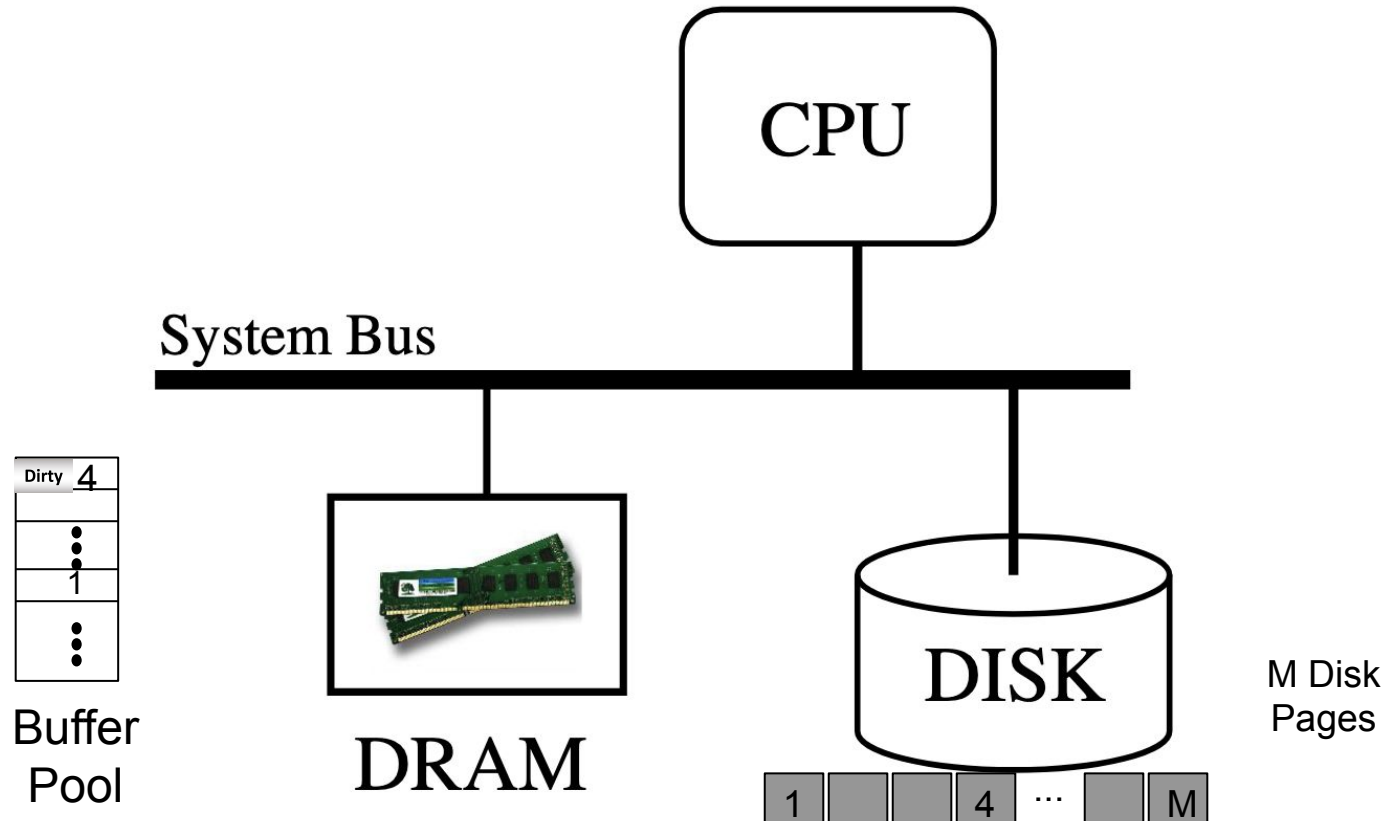
Buffer Pool Consists of F Disk Page Frames

Buffer pool manager returns the address of the frame containing Page P to the caller.



How to select a victim?

How does the buffer pool manager select a victim?



Least Recently Used (LRU)

- When the buffer pool is full and a new buffer page is required, the buffer pool manager evicts the page that has not been accessed for the longest time [1].

LRU (page p)

If p is in the buffer then $LAST(p) = \text{current time}$;

Else

i) $Min = \text{current time} + 1$;

ii) For all pages q in the buffer do

a) If $(LAST(q) < min)$

victim = q

$Min = LAST(q)$

iii) If victim is dirty then flush it to disk

iv) Fetch p into the buffer frame held by victim

v) $LAST(p) = \text{current time}$



Least Recently Used (LRU)

- When the buffer pool is full and a new buffer page is required, the buffer pool manager evicts the page that has not been accessed for the longest time [1].

LRU (page p)

If p is in the buffer then $LAST(p) = \text{current time}$;

Else

i) $Min = \text{current time} + 1$;

ii) For all pages q in the buffer do

a) If $(LAST(q) < min)$

victim = q

Min = $LAST(q)$

iii) If victim then flush it to disk

iv) Insert p into the buffer frame held by victim

$LAST(p) = \text{current time}$

Too expensive. Complexity is $O(F)$. How to implement efficiently, $O(1)$?



Fast Implementation of LRU

1. A doubly linked list of elements. Each element identifies a page k .
2. Every time a page is referenced, move the page to the end of the list.
3. Victim is always the head of the list.

How to implement Step 2?

- Maintain a head that points to the first element of the linked list, a tail that points to the last element of the linked list, and a hash table that indexes from page id to its element.

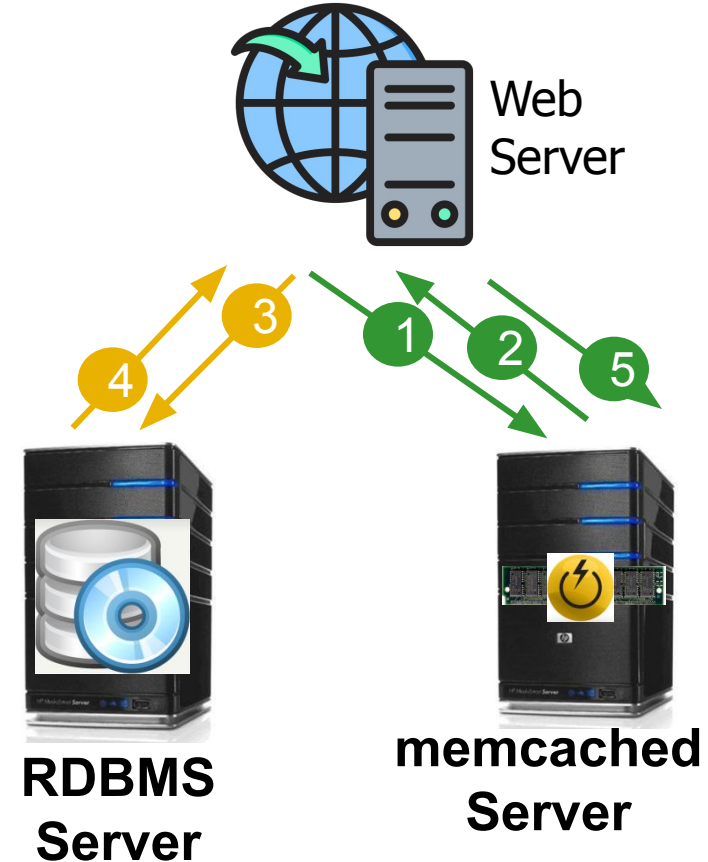
How to avoid the overhead of writing a dirty page back to the disk? Many possible solutions. Here are two:

- When the head points to a dirty page, schedule it to be written to disk asynchronously and select the next clean page on the list as the victim.
- Background threads that write dirty pages back to disk by iterating LRU's doubly linked list.



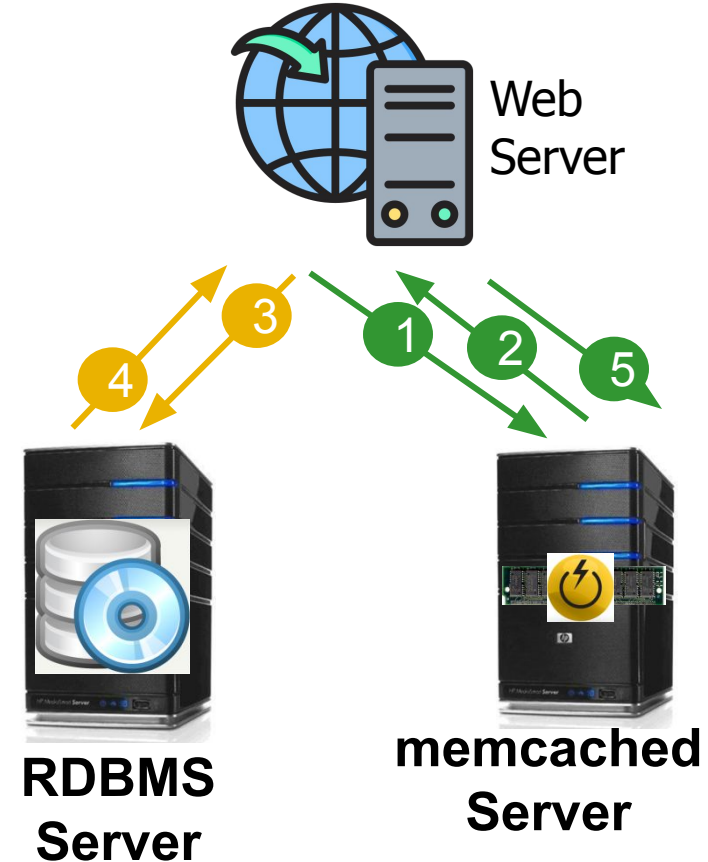
LRU: Key-Value Stores

- A page is similar to a key-value pair:
 - key is page-id
 - value is the content of the disk page
- LRU is implemented by cache managers such as memcached.
- Limitation: LRU decides what page to evict from memory based on too little information (time of last reference).
- What is missing?
 - Size of key-value pairs
 - Cost of key-value pairs, time to compute a key-value pair, e.g., simple query processing using index structures vs. complex analytics using time consuming joins and aggregates.



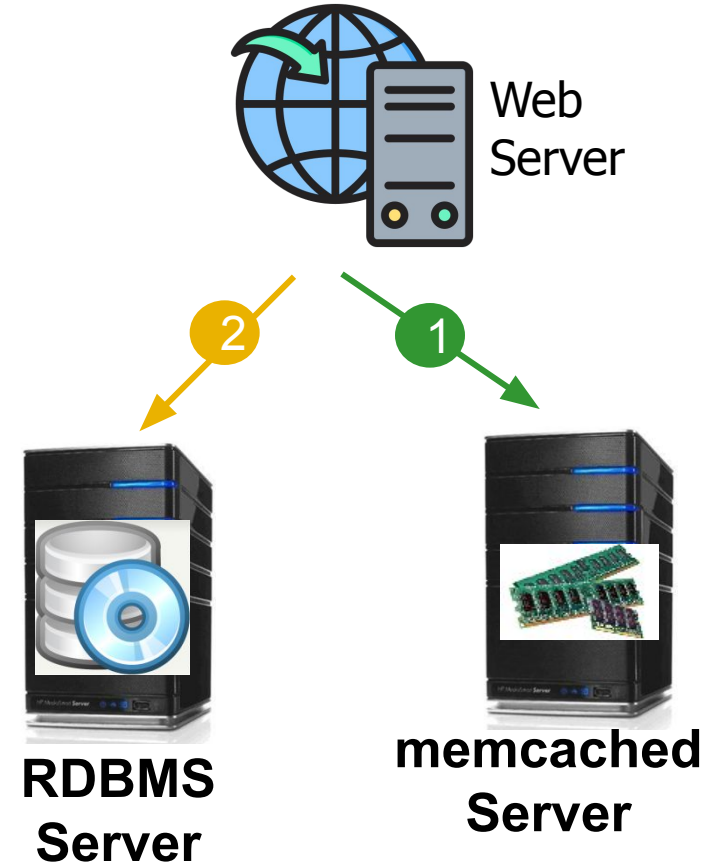
Cache Augmented Data Stores: Reads

1. Value = Get (Key)
2. If Value is found, go to Step 6.
3. SQL/Graph queries
4. Process query results
 - Application constructs Value using the results
5. Put(Key, Value)
6. Use Value to generate HTML result page



Cache Augmented Data Stores: Writes

- SQL DML Command:
Insert, Delete, Update
- Several paradigms:
Invalidate, Refresh (Refill),
and incremental update.
- Additional details on
November 10, 2025.



Key-Value Pairs with Varying Costs

- Millions of key-value pairs corresponding to different member profiles, each computed using a simple database look-up that executed in a few milliseconds.
- Thousands of key-value pairs computed using a machine-learning algorithm that processed Terabytes of data and required hours of execution.
- This processing time is one definition of the cost of a key-value pair.

Key-Value Pairs with Varying Costs

- Millions of key-value pairs corresponding to different member profiles, each computed using a simple database look-up that executed in a few milliseconds.
- Thousands of key-value pairs computed using a machine-learning algorithm that processed Petabytes of data and required days of execution.
- This processing time is one definition of the cost of a key-value pair.

Ideally: Do not evict key-value pairs with a very high cost unless those keys will not be accessed ever again.

Key-Value Pairs with Varying Costs

- Millions of key-value pairs corresponding to different member profiles, each computed using a simple database look-up that executed in a few milliseconds.
- Thousands of key-value pairs computed using a machine-learning algorithm that processed Petabytes of data and required days of execution.
- This processing time is one definition of the cost of a key-value pair.

Challenge: How does the system know a key-value pair will not be accessed again in the future?

Key-Value Pairs with Varying Costs

- Millions of key-value pairs corresponding to different member profiles, each computed using a simple database look-up that executed in a few milliseconds.
- Thousands of key-value pairs computed using a machine-learning algorithm that processed Petabytes of data and required days of execution.
- This processing time is one definition of the cost of a key-value pair.

LRU does not consider cost of key-value pairs. It considers only recency of references to the key-value pairs.

1. Human tuning
2. CAMP, Cost Adaptive Multi-queue eviction Policy

Solution 1: Human Tuning

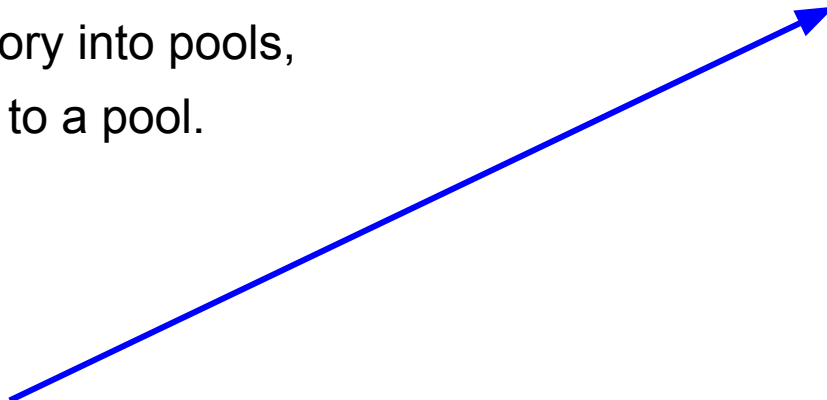
- A system administrator:
 - a. Identifies grouping of key-value pairs based on similar size, cost, or both,
 - b. Partitions the available memory into pools,
 - c. Assigns one or more groups to a pool.
- Each pool uses LRU.



Solution 1: Human Tuning

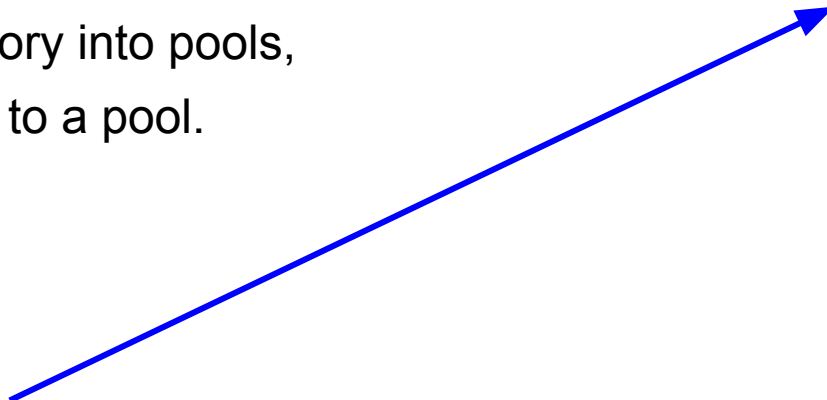
- A system administrator:
 - a. Identifies grouping of key-value pairs based on similar size, cost, or **both**,
 - b. Partitions the available memory into pools,
 - c. Assigns one or more groups to a pool.
- Each pool uses LRU.

What does this mean?



Solution 1: Human Tuning

- A system administrator:
 - a. Identifies grouping of key-value pairs based on similar size, cost, or **both**,
 - b. Partitions the available memory into pools,
 - c. Assigns one or more groups to a pool.
- Each pool uses LRU.



Use cost per byte: Cost/Size
This is cost to size ratio for a key-value pair.

Challenges:

- How does the system administrator:
 - a. Group trillions of key-value pairs across a dozen applications?
 - b. Partition memory?
 - c. Mapping of two or more groups to one partition? Why were they not assigned to the same group to begin with?
- What happens when the administrator does a poor job or the workload evolves quickly?



"Human Tuning" is partitioning resources. General is dividing his own army. It is not a winning strategy.

- Efficient implementation of the GreedyDualSize algorithm. It approximates GreedyDualSize.
 - a. A parameter controls its precision.
- Key features:
 - a. As efficient as LRU
 - b. Considers size, cost, and recency of references
 - c. Adapts to evolving access patterns
 - d. No partitioning of memory/resources

Algorithm 1 GreedyDualSize.

Initialize $L \leftarrow 0$.

Process each request for key-value pairs in turn.

The current request is for key-value pair p :

- (1) if p is already in memory (denoted by M),
 - (2) $L \leftarrow \min_{q \in M \setminus \{p\}} H(q)$.
 - (3) else,
 - (4) while there is not enough room in memory for p ,
 - (5) Evict the q with the smallest $H(q)$.
 - (6) $L \leftarrow \min_{q \in M} H(q)$.
 - (7) Bring p into memory.
 - (8) $H(p) \leftarrow L + \text{cost}(p) / \text{size}(p)$.
-

To understand GDS:

1. $H(p)$ approximates the value of having key-value p in memory. It is computed from a global variable L and the cost per byte of a key-value pair,
2. L is increasing in time.

Algorithm 1 GreedyDualSize.

Initialize $L \leftarrow 0$.

Process each request for key-value pairs in turn.

The current request is for key-value pair p :

- (1) if p is already in memory (denoted by M),
 - (2) $L \leftarrow \min_{q \in M \setminus \{p\}} H(q)$.
 - (3) else,
 - (4) while there is not enough room in memory for p ,
 - (5) Evict the q with the smallest $H(q)$.
 - (6) $L \leftarrow \min_{q \in M} H(q)$.
 - (7) Bring p into memory.
 - (8) $H(p) \leftarrow L + \text{cost}(p) / \text{size}(p)$.
-

To understand GDS:

1. $H(p)$ approximates the value of having key-value p in memory. It is computed from a global variable L and the cost per byte of a key-value pair,
2. L is increasing in time.



What happens when the cache first starts and is empty?

What happens when the cache is full and a cache resident item is referenced?

Algorithm 1 GreedyDualSize.

Initialize $L \leftarrow 0$.

Process each request for key-value pairs in turn.

The current request is for key-value pair p :

- (1) if p is already in memory (denoted by M),
 - (2) $L \leftarrow \min_{q \in M \setminus \{p\}} H(q)$.
 - (3) else,
 - (4) while there is not enough room in memory for p ,
 - (5) Evict the q with the smallest $H(q)$.
 - (6) $L \leftarrow \min_{q \in M} H(q)$.
 - (7) Bring p into memory.
 - (8) $H(p) \leftarrow L + \text{cost}(p) / \text{size}(p)$.
-

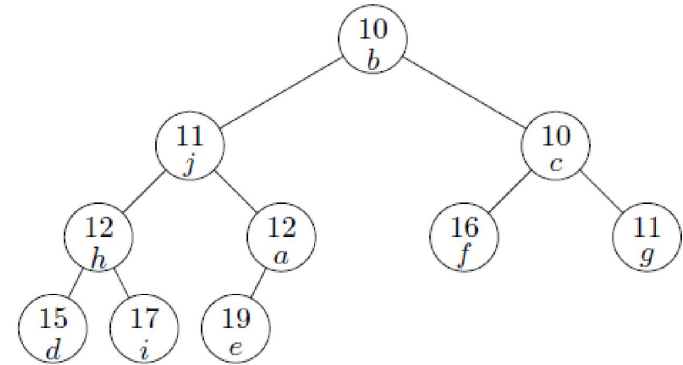
To understand GDS:

1. $H(p)$ approximates the value of having key-value p in memory. It is computed from a global variable L and the cost per byte of a key-value pair,
2. L is increasing in time.

A key-value p with $\text{cost}(p)/\text{size}(p)$ ratio that is c times that of another key-value pair q will remain in the cache c times longer than q .



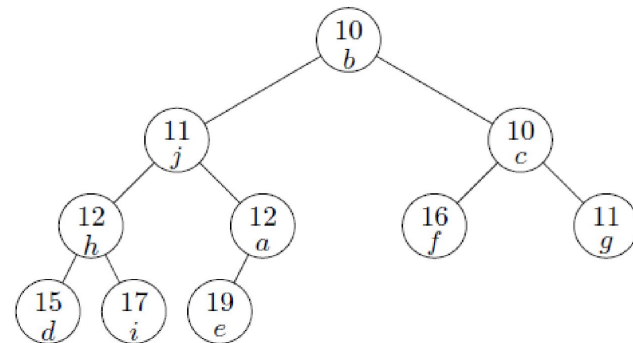
- Use a priority queue implemented as a heap:
 - A heap is a nearly complete binary tree implementation. The priority of any node in the tree is at most the priority of its children. All levels are filled except the lowest level, i.e., nearly complete.



- $H(p)$ is the priority of a key-value pair.
- Why is a priority queue implementation naive?

Why is Naive Bad?

- Worst-case performance:
 - Log n where n is the number of key-value pairs.
 - With Facebook, n is in the order of trillions.
- Limitation:
 - CPU time to maintain the heap becomes significant.
 - Heap is inherently single threaded.



Is GDS partitioning resources?



Is GDS partitioning resources? No!

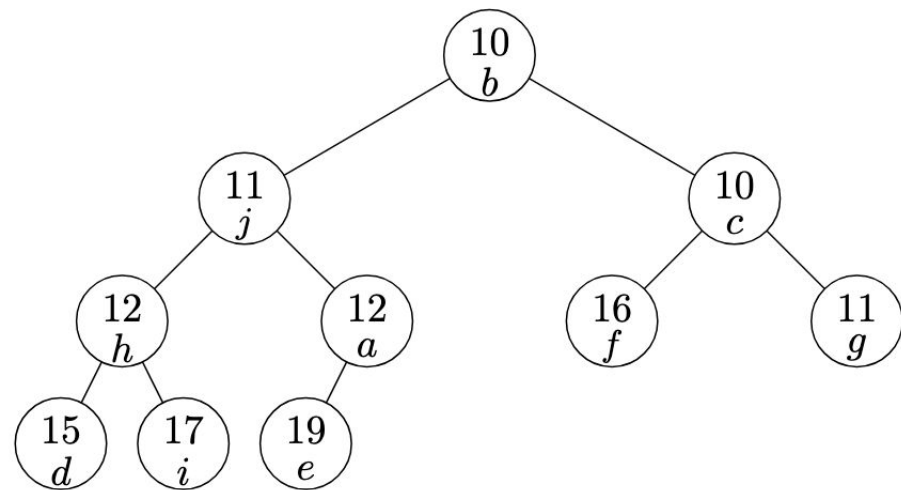


- GDS's insistence to evict the key-value pair with the absolute minimum priority H value is likely overkill. Instead, evict the key-value pair whose priority is approximately smallest.
- Step 1: CAMP rounds the priority for every key-value pair by rounding its cost/size ratio before adding it to L .
 - a. This results in a smaller set of possible cost/size values. Organize the key-value pairs with the same cost/size values in an LRU queue.
 - b. Maintain priority across the LRU queues using a heap. Evict from the LRU queue with the lowest priority.

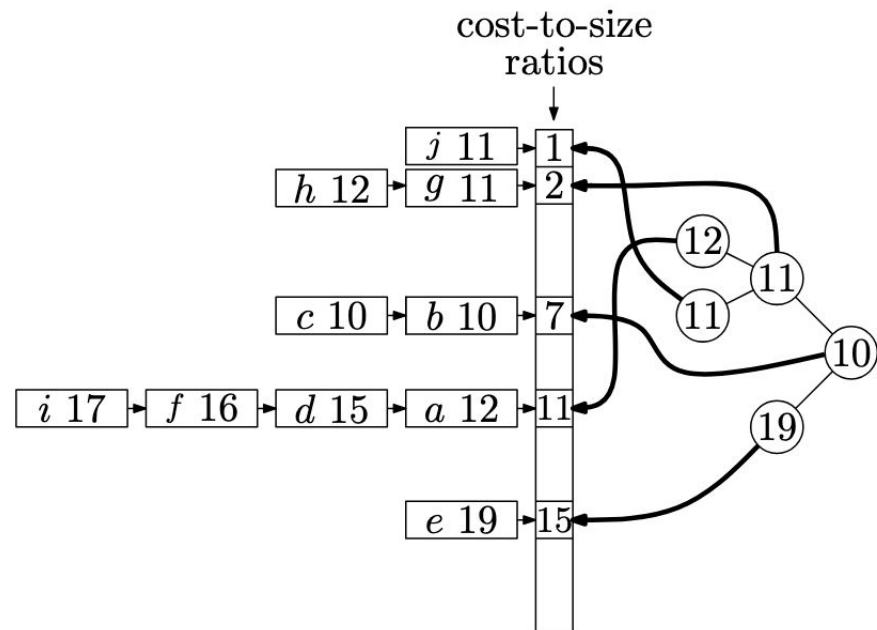
GDS vs. CAMP

CAMP groups key-value pairs with the same cost/size together using a LRU queue.

Heap is at the granularity of a grouping based on the priority of the head of LRU.



GDS's heap



CAMP's priority queue

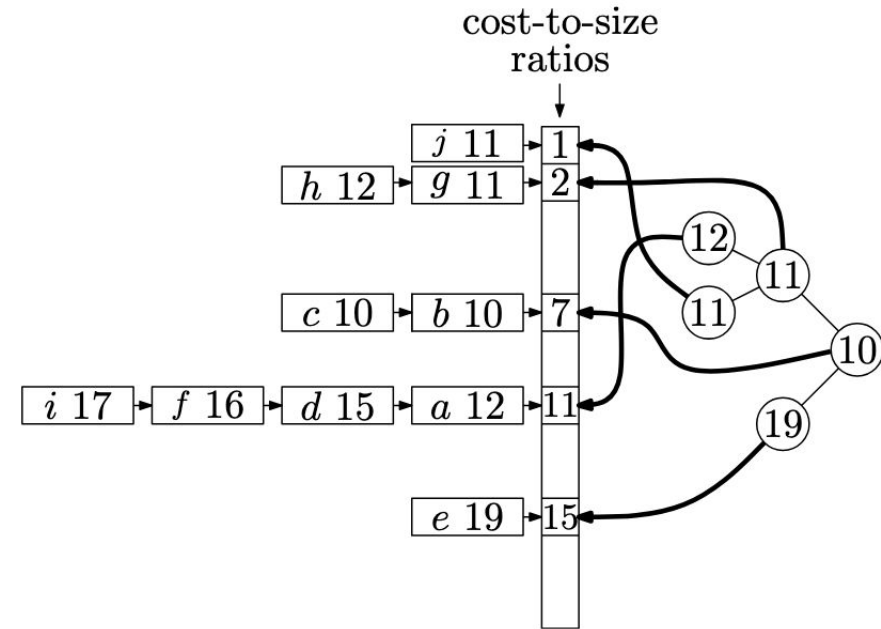
CAMP: Observations

One LRU queue for each cost/size ratio.

When a key-value is referenced, CAMP adjusts its priority and moves it to the tail of the LRU queue.

If key-value pairs within each queue are stored according to LRU then the key-value pairs are automatically ordered according to their priority.

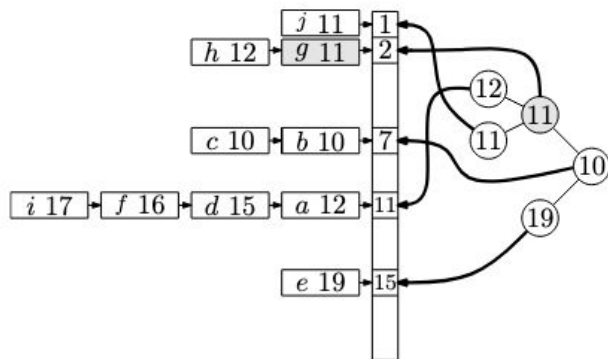
The first key-value pair in each queue has the lowest priority. It is a candidate victim.



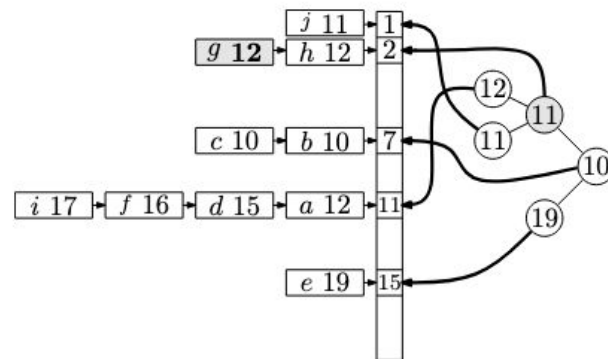
CAMP's priority queue

Consider a reference for key="g"

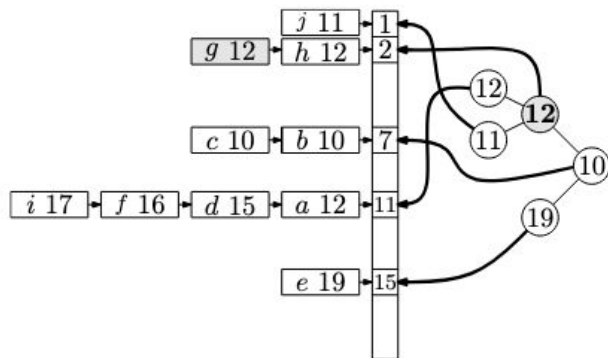
CAMP: Hit for g with L=10



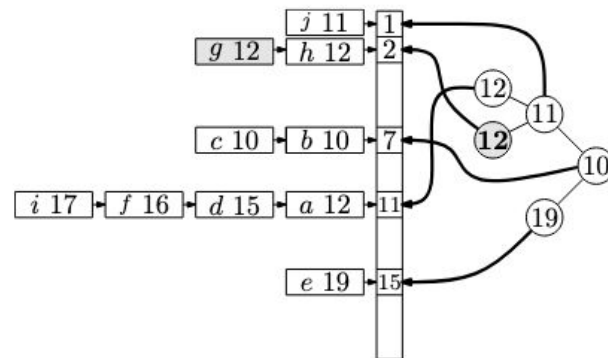
(a)



(b)

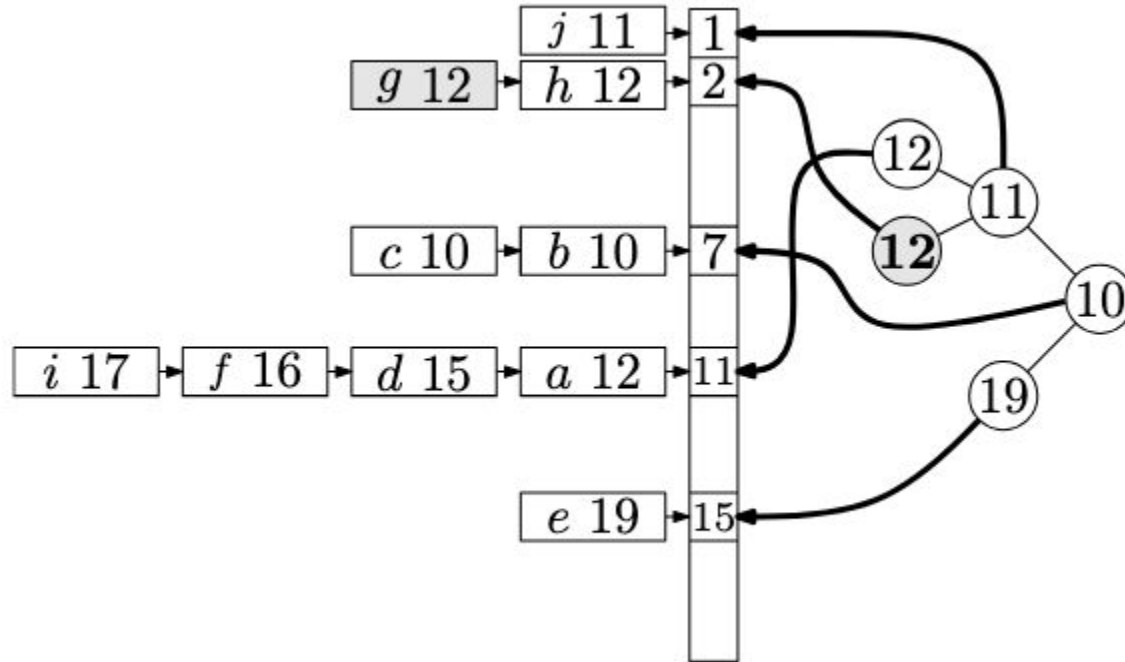


(c)



(d)

CAMP: Memory is Full, victim?

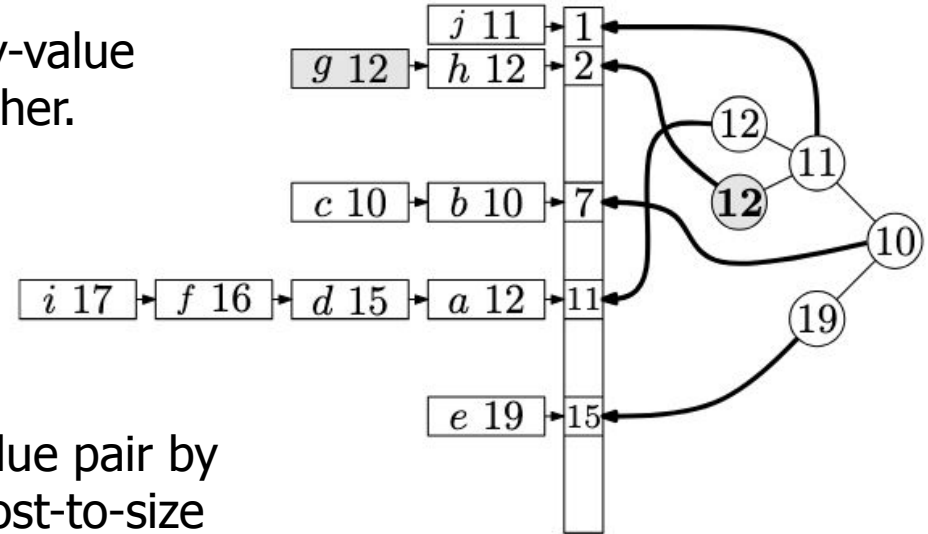


USC Viterbi
School of Engineering



CAMP: Requirements

1. Assign an integer priority H value to a key-value pair. cost/size is a fraction, a real number.
2. A rounding technique that groups key-value pairs with similar cost/size ratio together.



Solutions:

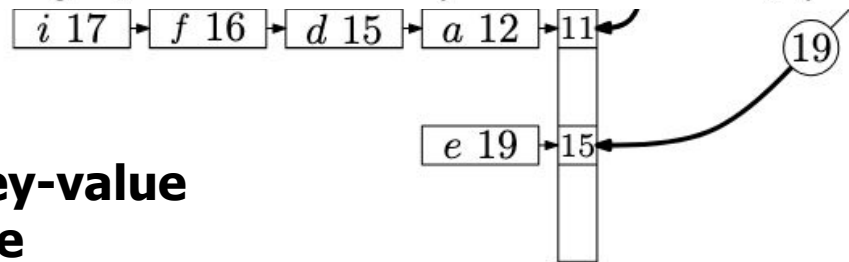
1. Divide the cost-to-size ratio of a key-value pair by a lower bound estimate on the smallest cost-to-size ratio that can ever occur.
2. Round the resulting integer using high order bits.

CAMP: Requirements



can solve this problem by first dividing the cost-to-size ratio by a lower bound estimate on the smallest cost-to-size ratio that can ever occur. Then we perform the actual rounding to the nearest integer. The cost of each key-value pair is a non-negative integer, so 1 divided by the maximum size of any key-value pair can serve as a lower bound for the cost-to-size ratio. Thus, we are effectively multiplying each cost-to-size ratio by the size of the largest key-value pair. Although we do not know the maximum size of a key-value pair *a priori*, we can determine it adaptively. (The CAMP.pdf paragraph describes why we do not simply use a

CAMP.pdf



Solutions:

1. **Divide the cost-to-size ratio of a key-value pair by a lower bound estimate on the smallest cost-to-size ratio that can ever occur.**

2. Round the resulting integer using high order bits.

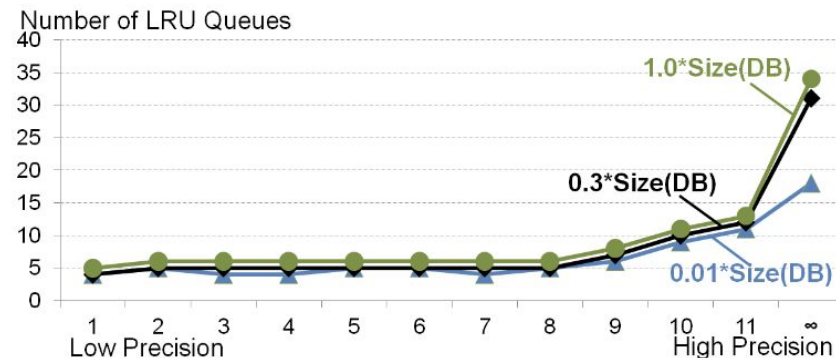
Objective: Assign key-value pairs with similar cost-to-size values to the same LRU queue AND values that have different orders of magnitude should remain distinct.

How? Preserve only the p most significant bits starting with b , the highest order bit that is not zero. If b is less than p then the value is not rounded.

regular rounding	CAMP's rounding
10110 1011 → 10110 0000	<u>1011</u> 0 1011 → 1011 0000
00101 0011 → 00101 0000	00 <u>1010</u> 011 → 00101 0000
00000 1010 → 00000 0000	00000 <u>1010</u> → 000001010
000000 111 → 00000 0000	000000 <u>111</u> → 000000111

Table 1: Rounding with (binary) precision 4

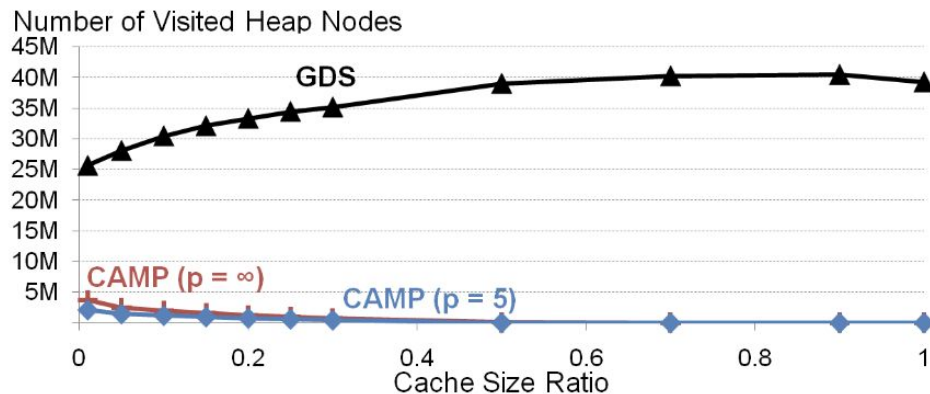
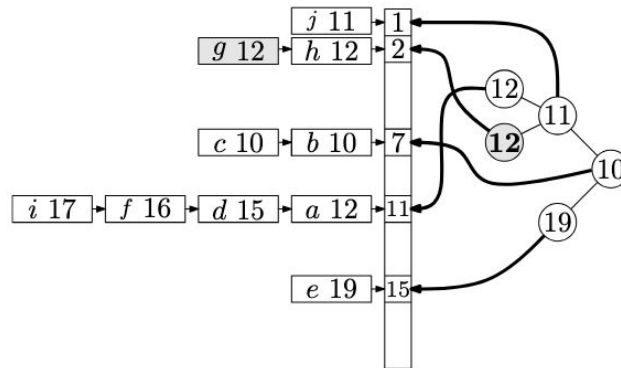
- A higher precision increases the number of LRU queues.
- With precision set to infinity, there are still LRU queues because of similarity in cost/size.



(b) Number of LRU queues maintained by CAMP with different precisions.

CAMP vs. GDS

- GDS updates its heap every time the priority of a key-value pair is updated. More memory means more key-value pairs to update.
- CAMP updates every time the priority of a key-value pair is updated. With larger memory, more items can be stored and there are fewer updates to the cache.



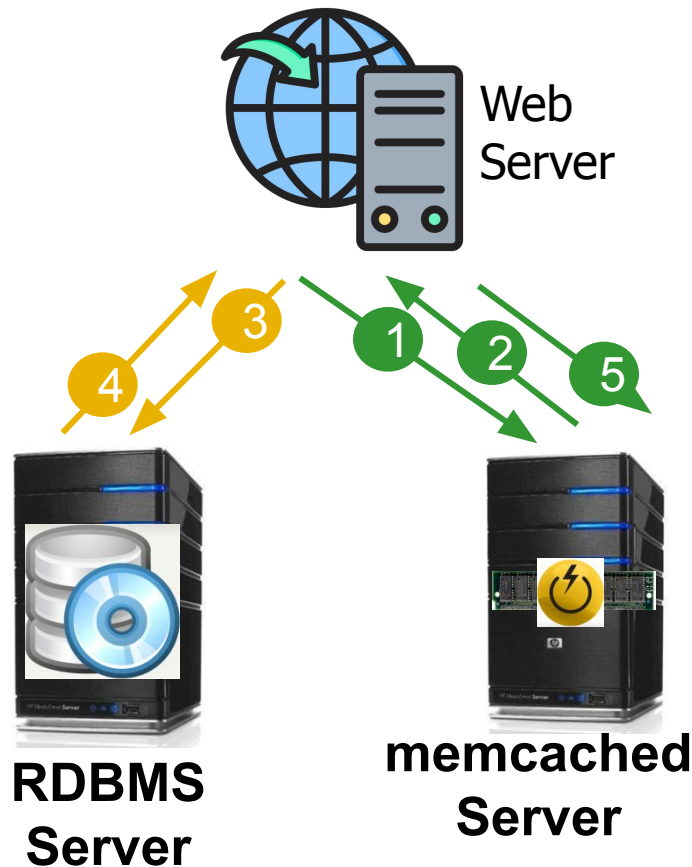
Cache Size Ratio = $\text{Size(Mem)}/\text{Size(DB)}$

Evaluation: Using BG

- Use BG to generate a trace of key-value references.
- Skewed pattern of access with 70% of requests reference 20% of keys.
- Cost is selected from $\{1, 100, 10K\}$. A key-value pair is assigned a cost with equal probability.
- Once a cost is assigned to a key-value pair, it remains in effect for the entire trace.

Metrics:

- Miss rate: Number of misses/total # of requests
- Cost-miss ratio: Sum the cost for each request that results in a cache miss/Sum of the costs for all the requests.



Evaluation: Using BG

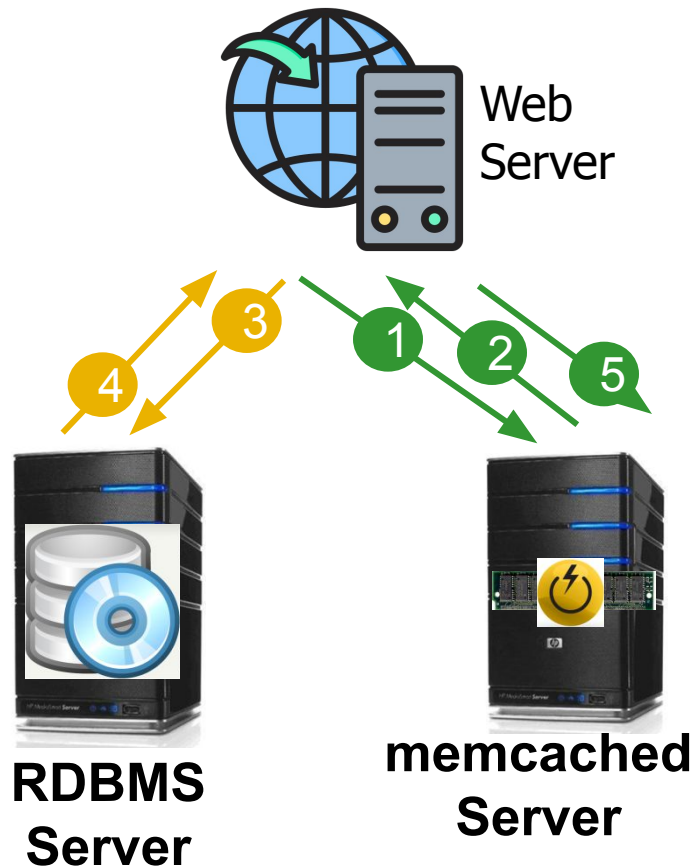
- Use BG to generate a trace of key-value references.
- Skewed pattern of access with 70% of requests reference 20% of keys.
- Cost is selected from {1, 100, 10K}. A key-value

What is the ideal value for the metrics?

- Once a cost is assigned to a key-value pair, it remains in effect for the entire trace.

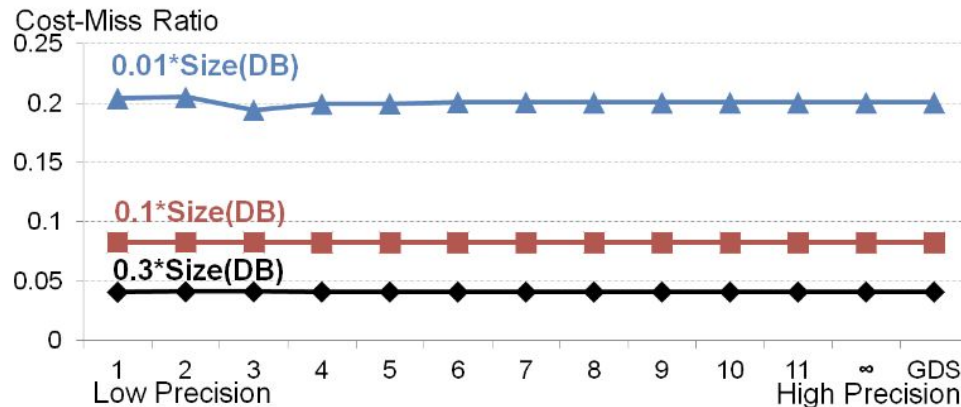
Metrics:

- Miss rate: Number of misses/total # of requests
- Cost-miss ratio: Sum the cost for each request that results in a cache miss/Sum of the costs for all the requests.



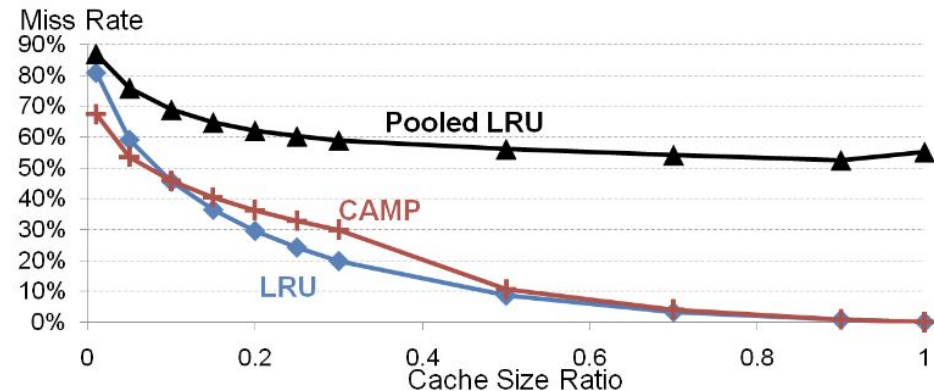
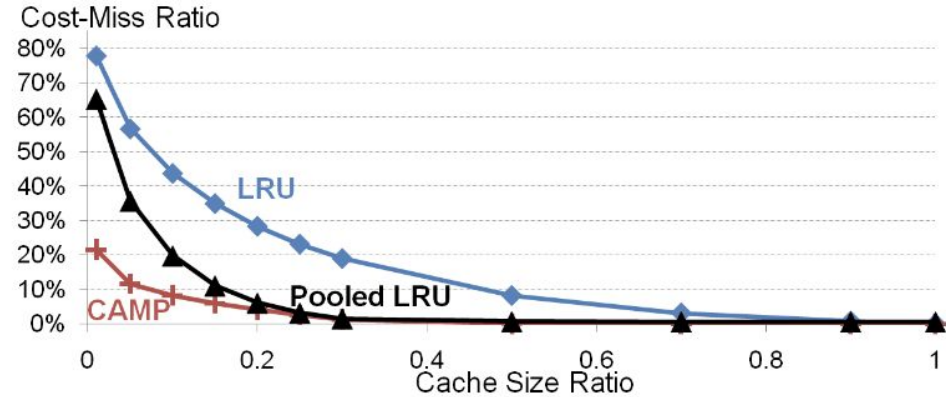
Lesson 1: Cost-Miss Ratio & Precision

- Cost-miss ratio is independent of the precision.
- The cost-miss ratio goes higher with smaller memory sizes, i.e., $0.01 * \text{size}(\text{DB})$ versus $0.3 * \text{size}(\text{DB})$



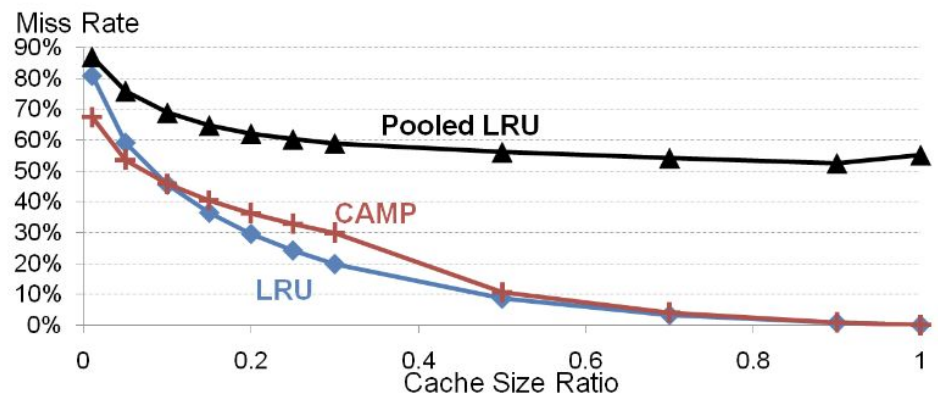
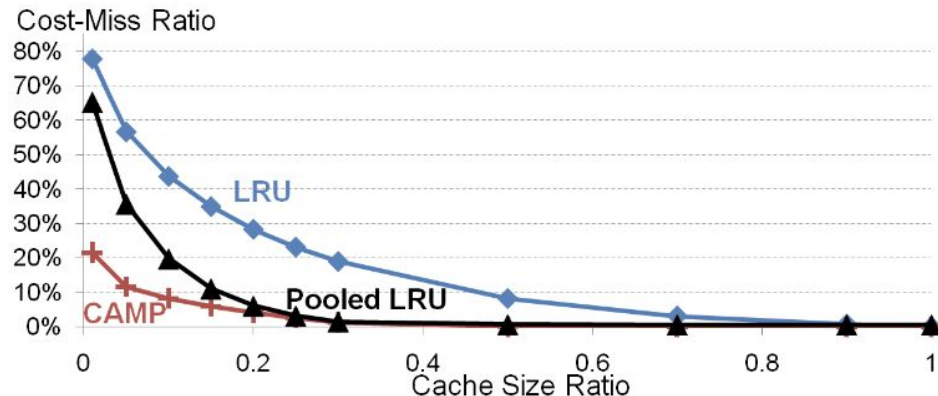
Lesson 2: CAMP is Superior to LRU

- CAMP provides a lower cost-miss ratio by considering cost. LRU does not consider cost; only recency of references.



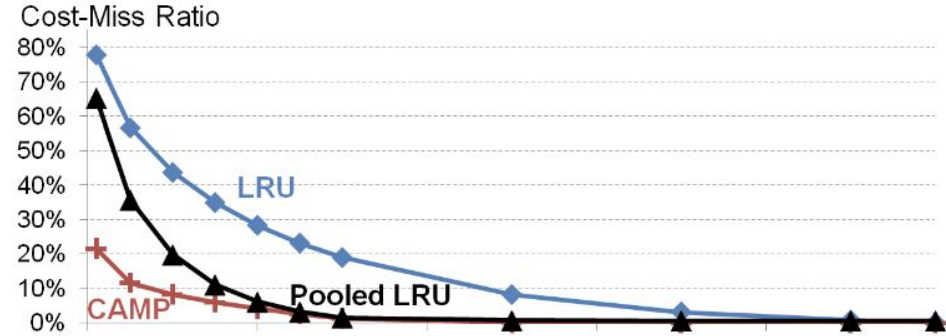
Lesson 2: CAMP is Superior to Pooled LRU

- Pooled LRU constructs 3 partitions of memory.
How?
 - Uniform versus Cost
- CAMP provides a lower cost-miss ratio when memory is scarce. Notice the higher miss rate of pooled LRU.

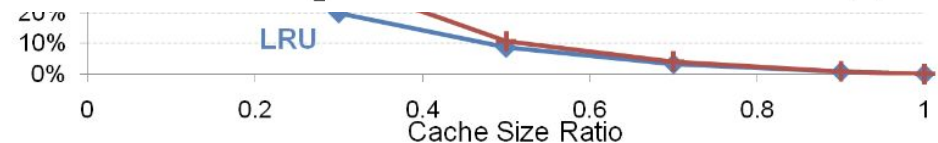


Lesson 2: CAMP is Superior to Pooled LRU

- Pooled LRU constructs 3 partitions of memory.
How?
 - Uniform versus Cost



is so close, that we only display the cost-miss ratio for LRU in Figure 5c. When memory is partitioned using cost, Pooled LRU improves over LRU's cost-miss ratio. Furthermore, it is able to match CAMP's cost-miss ratio when given a large enough cache size by assigning practically all of it to the most expensive pool. However, this improvement is at the expense of a significantly worse miss rate (see Figure 5d), since, even with a large cache size, the cheapest pool has nearly a 100% miss rate and the second pool has a miss rate of 65%.





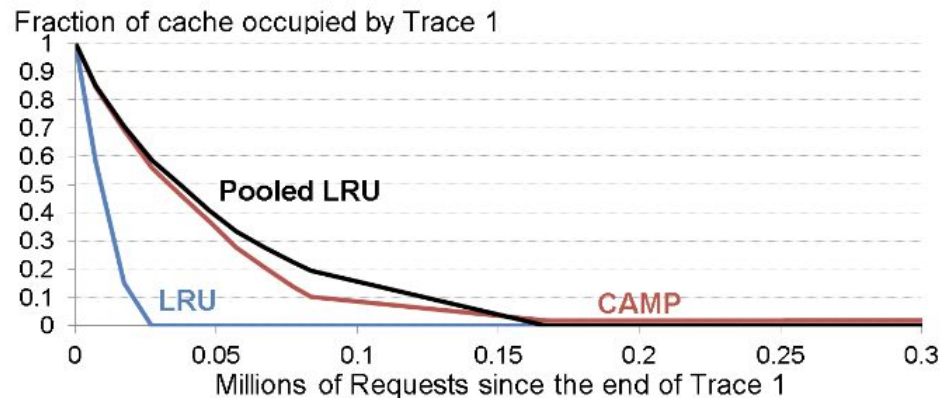
- Evolving workloads. How?
- 10 trace files each with different set of object references.
- Once the simulator switches from TF1 to TF2, none of the objects referenced by TF1 are referenced again.

Cost-miss ratio and miss rate trends remain the same as before.

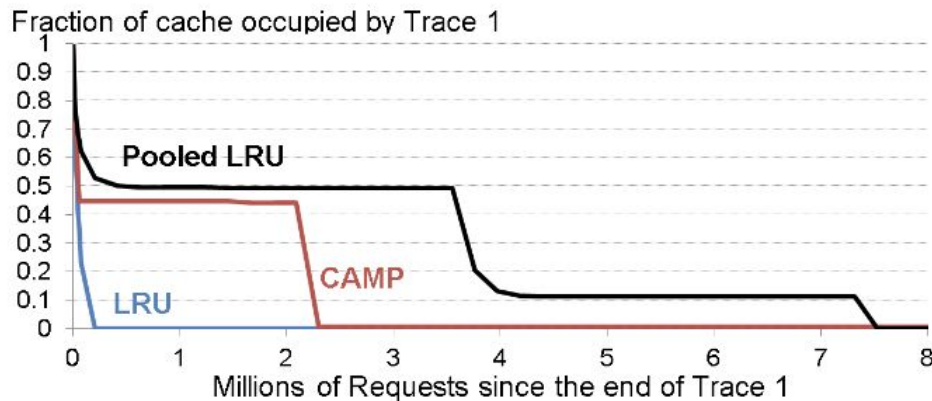
Lesson 3: CAMP Evolves



- Evolving workloads. How?
- 10 traces files each with different set of object references.
- Once the simulator switches from TF1 to TF2, none of the objects referenced by TF1 are referenced again.

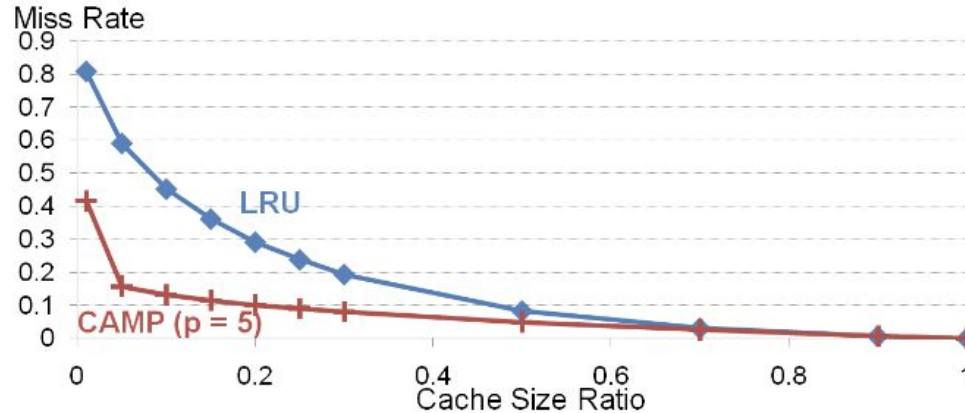


(c) Fraction of cache occupied by trace 1 items, with cache size ratio set at 0.25.



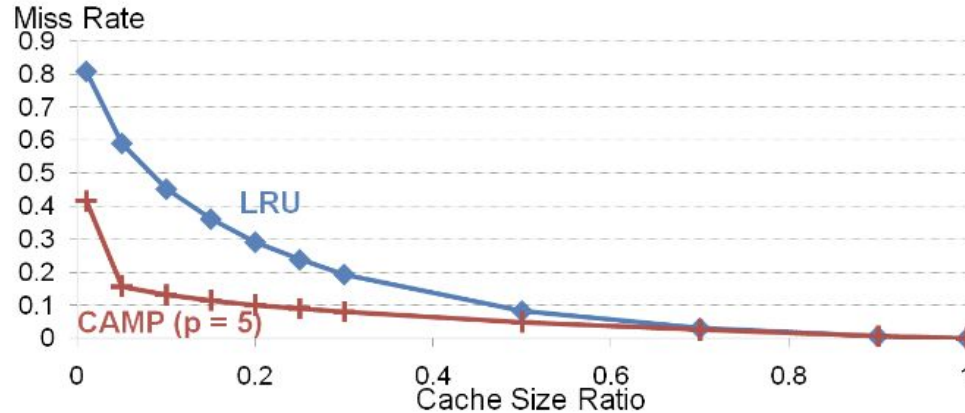
(d) Fraction of cache occupied by trace 1 items, with cache size ratio set at 0.75.

- With variable sized key-value pairs that have identical cost, CAMP renders small key-value pairs KVS resident. CAMP provides a lower miss rate when compared with LRU.



Pooled LRU constructs 1 pool because cost is fixed. Its behavior is same as LRU.

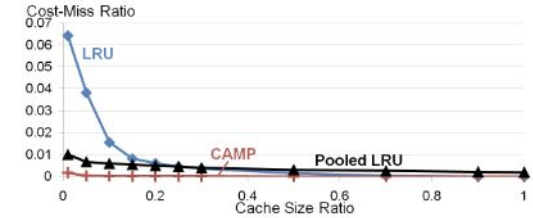
- With variable sized key-value pairs that have identical cost, CAMP renders small key-value pairs KVS resident. CAMP provides a lower miss rate when compared with LRU.



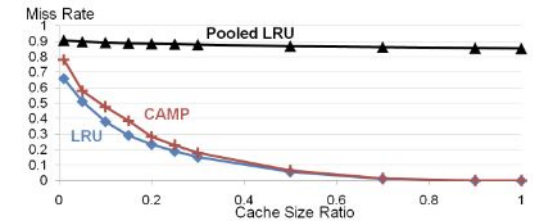
Pooled-LRU constructs one pool and behaves the same as LRU. Because the cost of the key-value pairs is set to 1, the cost-miss ratio of a technique is equal to its miss rate. Hence

It is very challenging to construct pools with Pooled LRU.

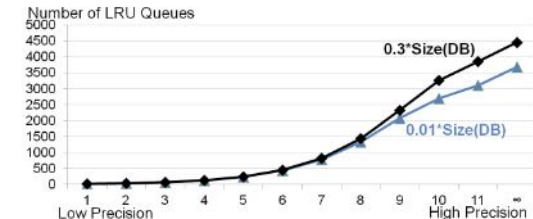
CAMP continues to provide a lower cost-miss ratio.



(a) Cost-miss ratio as a function of cache size ratio.

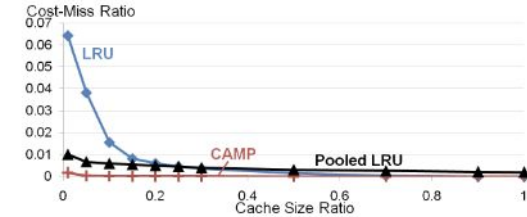


(b) Miss rate as a function of cache size ratio.

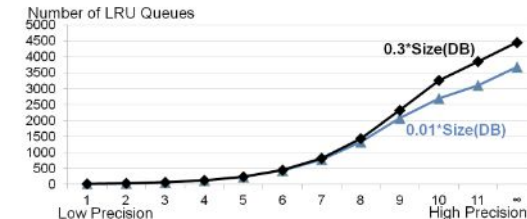


(c) Number of queues as a function of precision.

It is very challenging to construct pools with Pooled LRU.



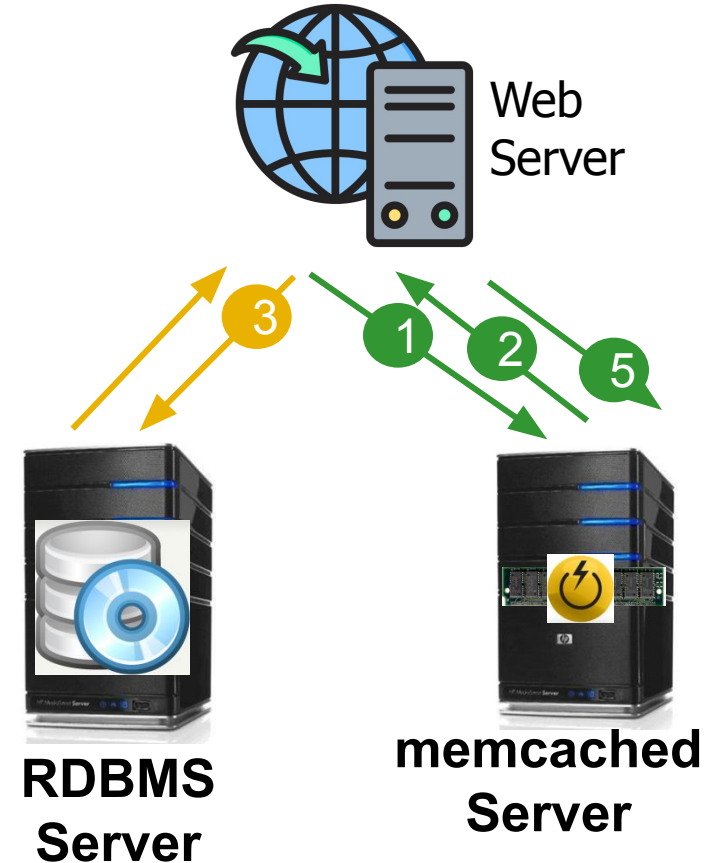
CA In comparison to the trace with three distinct cost values (Figure 5b), the trace with equi-sized key-value pairs has key-value pairs with many more distinct cost values. Therefore, for this trace, CAMP creates a larger number of LRU queues when no rounding takes place, see Figure 8c. This increase is due to the fact that there is a greater number of cost-to-size ratios as a result of the considerably larger set of cost values. With additional rounding, *i.e.* at lower precision, the number of LRU queues in the two traces decrease significantly and converge without any sizable performance degradation.



(c) Number of queues as a function of precision.

A Word of Wisdom

- When a cached key-value pair should be deleted then delete it. Just because it is not reachable, Do NOT depend on LRU or CAMP to evict it.



- CAMP is a memory management technique that considers cost, size, and recency of references to key-value pairs.
- Objective: Minimize cost-miss ratio.
- CAMP does NOT partition resources.
- CAMP adapts to evolving access patterns.